

Министерство науки и высшего образования Российской Федерации

**Федеральное государственное автономное образовательное
учреждение
высшего образования «Национальный исследовательский
университет
«Московский институт электронной техники»**

Лабораторная работа №4

**по дисциплине «Управление качеством программного
обеспечения»**

«Метрика Маккейба»

Москва, 2024

Для успешной сдачи лабораторной работы необходимо выполнить одно задание по варианту.

На 4:

Рассчитать Критерий 1 и Метрику Маккейба.

Нечётный номер компьютера - 1 вариант = 1-ое задание из раздела «задачи для самостоятельного решения».

Чётный номер компьютера - 2 вариант = 2-ое задание из раздела «задачи для самостоятельного решения».

На максимальный балл:

Рассчитать Критерий 1, Критерий 2 и Метрику Маккейба.

Особое внимание необходимо обратить на то, что расчёт второго критерия не заканчивается на вычислении цикломатического числа Z. Необходимо ещё посчитать S2.

Нечётный номер компьютера - 1 вариант = 7-ое задание из раздела «задачи для самостоятельного решения».

Чётный номер компьютера - 2 вариант = 9-ое задание из раздела «задачи для самостоятельного решения».

Задания делаются по образцу из примеров и оформляются в качестве отчётов. Перед сдачей необходимо внимательно проверить правильность выполнения, чтобы избежать понижения баллов.

Срок выполнения лабораторной работы с сохранением баллов – 2 недели.

Используемые языки: C, C++, C#, Java

ОЦЕНКА СТРУКТУРНОЙ СЛОЖНОСТИ ПРОГРАММ

2.1. Теоретические сведения

2.1.1. Критерии структурной сложности программ

Понятие структурной сложности программ

Структурная сложность программ определяется:

- количеством взаимодействующих компонентов;
- числом связей между компонентами;
- сложностью взаимодействия компонентов.

При работе программы многообразие ее поведения и разнообразие связей между ее входными и результирующими данными в значительной степени определяется *набором маршрутов*, по которым выполняется программа. При этом под маршрутом следует понимать чередование последовательностей вершин и дуг графа управления. Уже давно установлено, что *сложность программных модулей* связана не столько с размером программы, определяющей количество выполняемых команд, сколько с *числом маршрутов* ее исполнения и их *сложностью*.

При создании качественной программы основную сложность ее разработки определяют маршруты возможной обработки данных, которые должны быть тщательно проверены. Такую метрику сложности, связанную с анализом маршрутов, можно использовать для оценки трудоемкости тестирования и сопровождения модуля, а также для оценки потенциальной надежности его функционирования. Проведенные исследования подтверждают достаточно высокую адекватность использования структурной сложности программ для оценки

трудоемкости тестирования, вероятности ненайденных ошибок и затрат на разработку программных модулей в целом.

Совокупность маршрутов исполнения программного модуля условно можно разделить на две группы:

- вычислительные маршруты;
- маршруты принятия логических решений.

Группа *вычислительных маршрутов* объединяет в себе маршруты арифметической обработки данных и предназначена для непосредственного преобразования величин, являющихся элементарными результатами измерения каких-либо характеристик (переменных).

Для проверки вычислительных маршрутов можно сформировать достаточно простую стратегию их проверки. Во всем диапазоне преобразования входных переменных нужно выбрать несколько характерных точек, в которых проверяется работоспособность и корректность работы программы. К таким контрольным точкам можно отнести предельные значения, значения в точках разрыва, несколько промежуточных значений.

Сложность вычислительных маршрутов можно оценить следующей формулой:

$$S_1 = \sum_{i=1}^m \sum_{j=1}^{l_i} v_j, \quad (2.1.1)$$

где m — количество маршрутов исполнения программы; l_i — число данных, обрабатываемых в i -м маршруте; v_j — число значений обрабатываемых данных j -го типа ($2 \leq v_j \leq 5$).

Расчет сложности программы по этой формуле имеет высокую неопределенность из-за отсутствия конкретных требований в выборе количества значений переменной v_j при изменении входных данных. Поэтому применение этой формулы для оценки сложности весьма затруднительно.

В связи с тем что удельный вес (доля) вычислительной части во многих сложных программных комплексах обработки информации относительно невелика (общее число арифметических операций не выходит за пределы 5–10 %), вычислительные маршруты не играют доминирующей роли в определении структурной сложности программ. Это связано с тем, что проведение вычислений, даже весьма разветвленных, все же подчиняется определенным правилам, несущественно влияющим на логику проведения этих вычислений.

Группа *маршрутов принятия логических решений* объединяет пути, отражающие логику выполнения программы, которая может изменять последовательность выполнения команд, переводить управление на удаленные участки программного модуля или даже досрочно завершать его выполнение. Все эти факторы могут весьма значительно влиять на сложность программы.

Сложность маршрутов принятия логических решений оценивается формулой

$$S_2 = \sum_{i=1}^m p_i, \quad (2.1.2)$$

где p_i – число ветвлений или число проверяемых условий в i -м маршруте.

Поскольку большинство программных продуктов сочетает в себе как вычислительные маршруты, так и маршруты принятия логических решений, целесообразно установить способ определения общей сложности программы.

Общую сложность программы можно рассчитать по формуле

$$S = c \cdot \sum_{i=1}^m \left(\sum_{j=1}^{l_i} v_j + p_i \right), \quad (2.1.3)$$

где c – некоторый коэффициент пропорциональности, корректирующий взаимное использование метрик сложности вычислительных маршрутов и маршрутов принятия логических решений.

Критерии выделения маршрутов

Выделение маршрутов исполнения программы, которые необходимы для минимальной проверки всех возможных маршрутов ее выполнения и оценки структурной сложности, может осуществляться по различным критериям.

Наилучшим следует считать такой критерий, который позволит выделить все возможные маршруты исполнения программы при любых сочетаниях исходных и промежуточных данных. При этом формирование маршрутов зависит не только от структуры программы, но и от самих значений переменных в различные моменты времени и на различных участках выполнения программы. Такое выделение

маршрутов трудно формализовать, и оно представляется весьма трудоемким для оценки показателей сложности программной структуры.

Рассмотрим более простые критерии выделения маршрутов, учитывающие только структурные характеристики программных модулей. При этом будем анализировать *граф потока управления*, под которым понимается множество всех возможных путей исполнения программы, представленное в виде графа. Следовательно, *граф потока управления* – ориентированный граф, моделирующий поток управления программой (часто граф потока управления именуется *управляющим графом*). *Поток управления* – последовательность выполнения различных модулей и операторов программы.

Приведенные критерии для оценки сложности программных модулей в каждом случае характеризуют минимально необходимые способы проверок. Для проверки реальных программ этого количества проверок может оказаться недостаточно (например, циклы целесообразно проверять не однократно, а на нескольких промежуточных значениях и, кроме того, на максимальном и минимальном количествах исполнения циклов). Оценить достаточность проверок программы значительно труднее, так как кроме сложности структуры при этом необходимо анализировать сложность преобразования каждой переменной во всем диапазоне ее изменения и при сочетаниях с другими переменными.

Критерий 1

Данный критерий предполагает, что граф потока управления программой должен быть проверен по минимальному набору маршрутов, проходящих через каждый оператор ветвления по каждой дуге.

Прохождение по каждому маршруту осуществляется только один раз, повторная проверка дуг не проводится и считается избыточной. При этом в процессе проверки гарантируется выполнение всех передач управления между операторами программы и каждого оператора не менее одного раза. Следует заметить, что в настоящее время существуют алгоритмы, позволяющие автоматизировать процесс получения минимального множества маршрутов именно по этому критерию.

Структурная сложность программы по первому критерию вычисляется следующим образом:

$$S_i = \sum_{j=1}^m p_i, \quad (2.1.4)$$

где p_i – количество вершин ветвления в i -м маршруте без учета последней вершины.

Критерий 2

Критерий основан на анализе базовых маршрутов в программе, которые формируются и оцениваются на основе *цикломатического числа графа потока управления программы*.

Цикломатическое число Z исходного графа потока управления программой определяется формулой

$$Z = m - n + 2 \cdot p, \quad (2.1.5)$$

где m – общее число дуг в графе; n – общее число вершин в графе; p – число связанных компонентов графа.

Число связанных компонентов графа p равно количеству дуг, необходимых для превращения исходного графа в максимально связный граф. Максимально связным (или полносвязным) графом называется такой граф потока управления, у которого любая вершина доступна из любой другой вершины. Максимально связный (полносвязный) граф может быть получен из исходного графа потока управления программой путем замыкания конечной и начальной вершин. Для большинства корректных графов достаточно одной замыкающей дуги, проведенной между начальной и конечной вершинами. Под корректными понимаются такие графы потоков управления, у которых не содержится висячих и тупиковых вершин.

Второй критерий, выбранный по цикломатическому числу, требует однократной проверки или тестирования каждого *линейно независимого цикла* и каждого *линейно независимого ациклического участка* (т. е. не имеющего в своем составе циклических участков) программы.

Каждый линейно независимый ациклический маршрут или дикл отличается от всех остальных хотя бы одной вершиной или дугой, т. е. его структура не может быть полностью образована компонентами, входящими в состав других маршрутов.

При использовании второго критерия количество проверяемых маршрутов равно цикломатическому числу.

Для правильно структурированных программ характерно отсутствие циклов с несколькими выходами, такие программы не имеют переходов внутрь циклов или условных операторов, не имеют принудительных выходов из внутренней части циклов или условных операторов. Для таких программ цикломатическое число Z можно определить путем подсчета числа вершин n_v , в которых происходит ветвление.

Тогда цикломатическое число можно представить следующим образом:

$$Z = n_v + 1. \quad (2.1.6)$$

Анализ графов реальных программных модулей с достаточно большим фиксированным числом вершин показывает следующее:

- суммарная сложность тестовых маршрутов почти не зависит от детальной структуры графа и в основном определяется количеством ветвлений графа потока управления программы;

- при неизменном числе вершин в графах широкого профиля имеется большее количество маршрутов, чем в узких графах, но формируемые маршруты в среднем оказываются короткими.

Для автоматического анализа графов по второму критерию с помощью средств вычислительной техники используются матрицы смежности и достижимости графов, содержащие информацию о структуре проверяемой программы.

Матрица смежности – квадратная матрица, в которой единицы располагаются в позициях (i, j) , если в графе потока управления программой имеется дуга (i, j) . В противном случае, при отсутствии дуги в такой позиции, ячейки матрицы просто не заполняются, обозначая нулевое значение в соответствующей позиции. Пример матрицы смежности показан в табл. 2.1 для управляющего графа, приведенного на рис. 2.1, доведенного до вида полносвязного графа (пунктирная линия).

Матрицей достижимости называется квадратная матрица, в которой единицы располагаются в позиции, соответствующей дуге (i, j) . Пример матрицы достижимости имеет вид, показанный в табл. 2.2.

Матрица достижимости является основным инструментом для вычисления маршрутов по второму критерию. Используя компьютер, матрицу достижимости можно получить из матрицы смежности путем возведения ее в степень, величина которой равна числу вершин без последней в исходном графе потока управления.

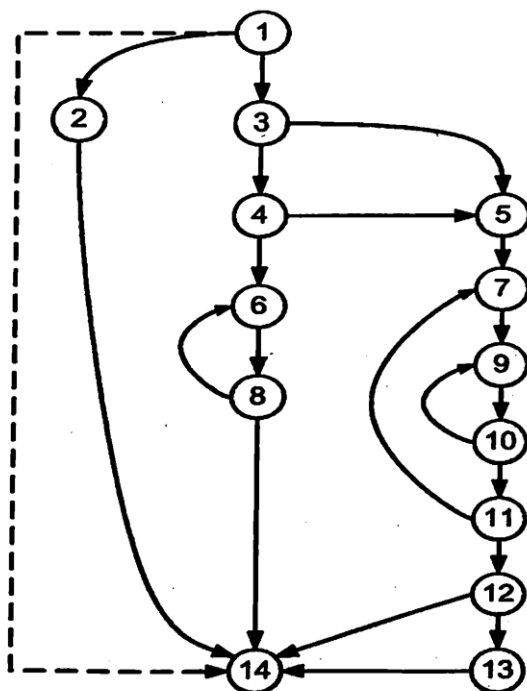


Рис. 2.1. Пример полностью управляющего графа

Таблица 2.1. Матрица смежности

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
1														
2	1													
3	1													
4			1											
5			1	1										
6				1			1							
7					1						1			
8						1								
9							1			1				
10									1					
11										1				
12											1			
13												1		
14		1						1				1	1	

Таблица 2.2. Матрица достижимости

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
1														
2	1													
3	1													
4	1		1											
5	1		1	1										
6	1		1	1		1		1			...			
7	1		1	1	1		1		1	1	1			
8	1		1	1		1		1						
9	1		1	1	1		1		1	1	1			
10	1		1	1	1		1		1	1	1			
11	1		1	1	1		1		1	1	1			
12	1		1	1	1		1		1	1	1			
13	1		1	1	1		1		1	1	1	1		
14	1	1	1	1	1	1	1	1	1	1	1	1	1	

С помощью матрицы достижимости можно сравнительно просто выделить циклы, отмечая диагональные элементы, равные единице, и идентичные строки. В табл. 2.2 для наглядности применена тонировка ячеек различными уровнями серого цвета. Для выделения вершин, входящих в состав различных циклов, использованы разные шрифты (полужирный и курсив).

Критерий 3

Более сильные критерии проверки и определения сложности структуры программы включают требования однократной проверки не только линейно независимых, но и всех линейно зависимых циклов и ациклических маршрутов. К таким критериям относится третий критерий формирования маршрутов для тестирования программ.

Третий критерий выделения маршрутов основан на формировании полного состава базовых структур графа потока управления программой и заключается в анализе каждого из реальных ациклических маршрутов исходного графа программы и каждого цикла, достижимого из всех этих маршрутов. При этом каждый из указанных компонентов структуры программы должен быть проанализирован хотя бы один раз.

Если при прохождении какого-либо ациклического маршрута исходного графа потока управления достижимы несколько элементарных циклов, то при тестировании должны исполняться все достижимые циклы.

Метрика Маккейба

Наиболее популярной метрикой, позволяющей оценить структурную сложность программных средств, является метрика Маккейба, построенная не на анализе лексических характеристик программы, а на результатах анализа потока управления от одного оператора к другому. Это позволяет (в отличие от метрик Холстеда) учесть логику построения программы при оценке ее сложности. На основе разработанных методов оценки сложности программ автор предложил стратегию проверки корректности ПС, которая получила название *основного маршрута тестирования* Маккейба.

Программное средство (алгоритм, спецификация) должно быть представлено в виде управляющего ориентированного графа

$$G = (V, E)$$

с V вершинами и E дугами, где вершины соответствуют операторам, а дуги характеризуют переход управления от одного оператора к другому.

Граф, описывающий программу в виде вершин-операторов и дуг-переходов, называют *графом потока управления* или *управляющим графом* программы.

Безусловно, для представления программы в виде графа необходимы определенные соглашения, определяющие положения о том, что считать узлом графа, так как синтаксис операторов в различных языках программирования может существенно отличаться. При этом обычно учитывают только исполнимые операторы и исключают группы операторов, назначением которых является описание данных. Желательно выбирать такие синтаксические формы операторов, которые в наибольшей степени подходят для отображения в виде узла графа. Линейные участки программы вообще можно заменить одним узлом графа, относя к нему группы операторов, выполнение которых осуществляется в прямой последовательности без каких-либо условий (это характерно для участков программ, содержащих последовательность прямых вычислений переменных). В этом отношении использование алгоритмов в схемном представлении или описание программ на псевдокоде может оказаться более удобной формой описания программы, чем текст на языке программирования. При любом варианте представления анализируемой программы желательно преобразовать операторы цикла в эквивалентную последователь-

ность операторов ветвления, добавив, при необходимости, операторы подсчета числа повторений цикла с «верхним» или «нижним» окончанием (счетчики циклов).

Метрика Маккейба характеризует цикломатическое число графа потока управления программой и определяется следующим выражением:

$$M = m - n + 2, \quad (2.2.1)$$

где m – количество ребер графа, n – число вершин графа.

Величину M , рассчитанную по этой формуле, называют *цикломатическим числом Маккейба*.

Цикломатическая сложность программы – структурная (или, иначе говоря, топологическая) мера сложности программы, применяемая для измерения качества ПО и основанная на методах статического анализа кода ПС. Цикломатическая сложность программы равна цикломатическому числу графа программы, увеличенному на единицу.

При вычислении цикломатической сложности используется граф потока управления программой: узлы графа соответствуют неделимым группам команд программы и ориентированным ребрам, каждый из которых соединяет два узла и соответствует двум командам, вторая из которых может быть выполнена сразу после первой. Цикломатическая сложность может также быть применена для отдельных функций, модулей, методов или классов в пределах анализируемого программного средства. Эта стратегия тестирования называется *основным маршрутом тестирования* Маккейба. Тестирование представляет собой проверку каждого линейного независимого маршрута графа программы. При такой проверке программы количество тестов должно быть равно ее цикломатической сложности.

Цикломатическая сложность части программного кода – счетное число линейно независимых маршрутов, проходящих через программный код. Например, если исходный код не содержит никаких точек решений, таких как указания условий IF или границы циклов (например, FOR), то сложность должна быть равна единице, поскольку есть только единственный маршрут выполнения программного кода. Если в тексте программы размещен единственный оператор IF, содержащий простое условие, то должно быть два пути выполнения кода: один путь через оператор IF с оценкой выполнения условия как ИСТИНА (TRUE) и другой – для альтернативной ветви при значении ЛОЖЬ (FALSE), в котором заданное условие не выполняется (рис. 2.1).

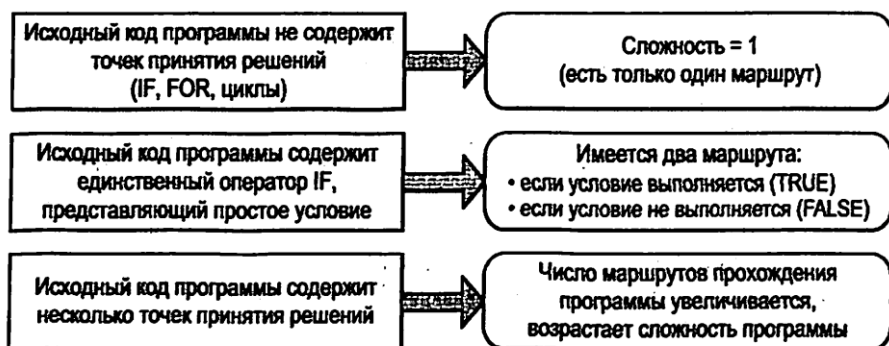


Рис. 2.2. Характеристика влияния точек принятия решений на сложность программы

Теоретической основой определения цикломатического числа Маккейба является теория графов. В теории графов цикломатическое число ориентированного графа определяется выражением

$$Z = m - n + 2 \cdot p, \quad (2.2.2)$$

где m – количество ребер, n – число вершин, p – количество компонентов связности графа.

Число компонентов связности p можно рассматривать как минимально необходимое количество ребер, которые нужно добавить к графу, чтобы сделать его полносвязным. Полносвязным считается граф, у которого существует путь из любой вершины графа в любую другую вершину графа. Как показал анализ достаточно большого количества управляющих графов, для обеспечения полносвязности графа достаточно добавления одной фиктивной дуги (которая отсутствует в реальном графе программы, потому и считается фиктивной) из его конечной вершины в начальную. Поэтому можно считать, что практически для любого графа управления программой число компонентов связности равно единице, т. е. $p = 1$. Подстановка значения $p = 1$ в формулу определения цикломатического числа дает значение цикломатического числа Маккейба.

Цикломатическое число определяет количество независимых контуров в полносвязном графе и, как следствие, количество различных маршрутов, ведущих из начальной вершины в конечную. При оценке сложности программы с применением цикломатического чис-

ла Маккейба справедливо следующее положение: если значение цикломатического числа больше 10, это означает, что программа обладает излишней сложностью и ее следует разбить на составные части (независимые модули или подпрограммы), для которых цикломатическое число будет иметь меньшее значение.

2.1.2. Особенности построения управляющих графов

Рассмотрим методику построения графов управления для типовых блоков, реализуемых основными операторами языков программирования. Напомним, что ориентированный управляющий граф позволяет учесть логику программы, обеспечивая возможность анализа потока передачи управления между операторами. Этот граф формируется из вершин, которые соответствуют операторам программы, и дуг, задающих переходы между этими операторами.

Прежде чем рассматривать типовые блоки программных модулей, отметим несколько общих замечаний, справедливых практически для любых программных блоков.

1. При построении графа управления программой надо учитывать только исполнимые операторы, не учитывая операторы описания.

2. Если в программе существует несколько операторов, не изменяющих порядок действий в программе, и они следуют один за другим, их допускается объединять в одну вершину графа.

3. При построении графа управления операторы цикла следует заменить несколькими вершинами. Например, для оператора типа *for* должны быть вершины, соответствующие операторам начального присваивания счетчика цикла, его изменению и ветвлению, определяющим продолжение или завершение выполнения цикла.

4. Аналогично несколькими вершинами заменяют и другие операторы цикла. Они должны соответствовать действиям, предшествующим циклу, определяющим продолжение или прекращение выполнения цикла, а также группе операторов, составляющих тело цикла — множеству повторяемых действий. Таким образом, в графе должны быть три группы вершин, определяющих три элемента, составляющих любой цикл (начало цикла, тело цикла, ветвление при окончании цикла — завершение или возвращение к исходному оператору). Далее это будет рассмотрено на конкретных примерах.

Правильно построенный управляющий граф позволяет оценить структурную (или топологическую) сложность программы, определяемую количеством независимых контуров в полносвязном графе.

Приведенные далее фрагменты исходных текстов программ, на основе которых будут рассматриваться особенности построения управляющих графов, написаны на разных языках программирования (C, C++ и C#), однако методика построения графов в равной степени применима к любым языкам программирования.

Линейная последовательность операторов

Рассмотрим фрагмент программы вычисления площади прямоугольника по длинам двух его сторон и приведем ее граф управления. Пример реализации такой программы на языке C# (приводятся только исполнимые операторы консольного приложения) приведен на рис. 2.3, а граф такого фрагмента показан на рис. 2.4.

Номера строк	Строки программы
1	<code>a=double.Parse(Console.ReadLine());</code>
2	<code>b=double.Parse(Console.ReadLine());</code>
3	<code>p=a*b</code>
4	<code>Console.WriteLine("{0:f}", p);</code>

Рис. 2.3. Фрагмент программы вычисления площади прямоугольника

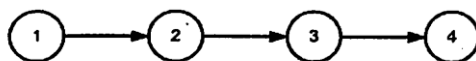


Рис. 2.4. Управляющий граф линейной последовательности операторов

Расчеты по такому графу управления программой будут такими:

- количество дуг $m = 3$,
- количество вершин $n = 4$,
- цикломатическое число Маккейба

$$Z = m - n + 2 = 3 - 4 + 2 = 1.$$

Заметим, что текст программы снабжен номерами, которые соответствуют номерам вершин графа. В дальнейшем будем и все другие фрагменты программы снабжать такими номерами.

Решение такой задачи можно записать по-другому, например так, как показано на рис. 2.5.

Номера строк	Строки программы
1	<code>a=double.Parse(Console.ReadLine());</code>
1	<code>b=double.Parse(Console.ReadLine());</code>
2	<code>Console.WriteLine("{0:f}", p=a*b);</code>

Рис. 2.5. Сокращенный вариант фрагмента-программы

Для такой программы граф управления приведен на рис. 2.6.

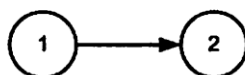


Рис. 2.6. Граф управления сокращенной линейной последовательностью операторов

Такой граф даст следующие результаты расчета характеристик:

- количество дуг: $m = 1$,
- количество вершин $n = 2$,
- цикломатическое число Маккейба

$$Z = m - n + 2 = 1 - 2 + 2 = 1.$$

Таким образом, цикломатическое число Маккейба осталось таким же, как и в первом случае. Этот пример подтверждает сделанное ранее утверждение о том, что последовательность выполняемых подряд операторов может быть объединена в одну вершину с одним входом и одним выходом, если они не изменяют последовательности выполнения программы.

Простое ветвление (оператор if)

Примеру программы с одним разветвлением может соответствовать задача вычисления максимального значения двух чисел и присвоения его результирующей переменной. В формализованном виде данную задачу можно записать следующим образом: $c = \max\{a, b\}$.

Фрагмент реализации задачи на языке программирования C запишется, например, так, как показано на рис. 2.7.

Номера строк	Строки программы
1	<code>scanf("%f%f",&a,&b);</code>
2	<code>if (a>b)</code>
3	<code>c=a</code>
4	<code>c=b;</code>
5	<code>printf("%f",c);</code>

Рис. 2.7. Фрагмент программы с разветвлением по оператору *if*

Граф управления для такого фрагмента приведен на рис. 2.8.

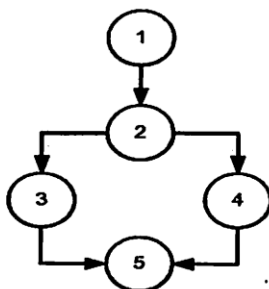


Рис. 2.8. Граф управления программы с разветвлением по оператору *if*

Характеристиками графа сложности такого фрагмента программы будут следующие показатели:

- количество дуг $m = 5$;
- количество вершин $n = 5$;
- цикломатическое число Маккейба

$$Z = m - n + 2 = 5 - 5 + 2 = 2.$$

Переключатель с множественным выбором

Такую функцию в программах реализуют операторы *switch*, *select*, *case* и им подобные.

Рассмотрим фрагмент программы, которая решает такую задачу.

В программу вводится символ. Необходимо определить, является ли такой символ допустимым для записи числа в римской системе счисления. Для упрощения примера будем распознавать не все сим-

волы, используемые в римской системе счисления, а примем к анализу только три: *I*, *V* и *X*. Фрагмент программы для решения такой задачи на языке C++ представлен на рис. 2.9.

Номера строк	Строки программы
1	cout<<"\n\n Input symbol I, V, X\n "; cin>>s;
2	switch (s)
	{
	case 'I':
3	cout<<" I – good symbol "; break;
	case 'V':
4	cout<<" V – good symbol "; break;
	case 'X':
5	cout<<" X – good symbol "; break;
	default:
6	cout<<" BAD SYMBOL ";
	}
7	getch();

Рис. 2.9. Фрагмент программы с разветвлением по оператору *if*

Граф потока управления этим фрагментом программы приведен на рис. 2.10.

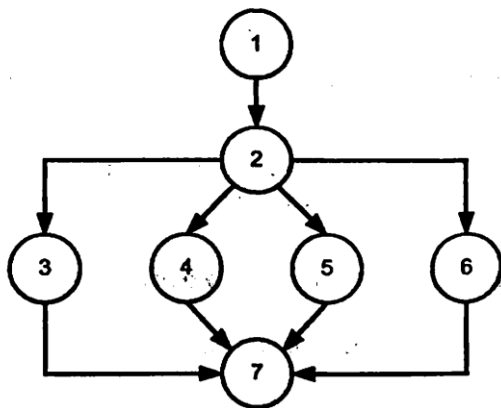


Рис. 2.10. Граф управления с множественным выбором

Количество дуг этого графа $m = 9$, а количество вершин $n = 7$. Рассчитаем для него цикломатическое число Маккейба:

$$Z = m - n + 2 = 9 - 7 + 2 = 4.$$

Отметим, что для тестирования такого фрагмента программы достаточно 4 теста (например, со следующими введенными значениями переменной s : I, V, X, v).

Заметим, что в графе на рис. 2.10 введена одна фиктивная вершина. Она соответствует участку программы, куда передается управление после завершения оператора *switch*. Возможно, введение дополнительного оператора пояснит эквивалентная форма записи приведенного выше фрагмента программы, приведенная на рис. 2.11.

Номера строк	Строки программы
1	cout<<"\n\n Input symbol I, V, X\n "; cin>>s;
2	switch (s)
	{
	case 'I':
3	q='I'; cout<<" I – good symbol "; break;
	case 'V':
4	q='V'; cout<<" V – good symbol "; break;
	case 'X':
5	q='X'; cout<<" X – good symbol "; break;
	default:
6	cout<<" BAD SYMBOL "; q=s;
	}
7	cout<<q;

Рис. 2.11. Фрагмент программы с разветвлением по оператору *if*

В этом фрагменте оператором *switch* дополнительной переменной q присваивается значение введенной переменной, а после выполнения этого оператора выполняется вывод присвоенного значения.

Программы с операторами цикла

Пусть имеется программа, вычисляющая сумму первых N членов натурального ряда

$$S = \sum_{i=1}^N i.$$

Фрагмент программы для вычисления этой суммы на языке C# может выглядеть так, как показано на рис. 2.12.

Номера строк	Строки программы
1	N = 5;
2	S = 0;
3	i = 1;
4	do
5	{
6	S += i;
7	i++;
8	} while (i<=N);
9	Console.WriteLine("s = {0:d}", s);

Рис. 2.12. Фрагмент программы вычисления суммы членов натурального ряда

Для построения графа потока управления при оценке структурной сложности программы надо заменить оператор цикла программы на эквивалентную последовательность операторов. Среди последних должны быть операторы присваивания и вычисления условий. Они должны отразить три элемента, присущих любому циклу:

- начальные присваивания до выполнения цикла;
- повторяющиеся действия (тело цикла);
- условия окончания (или продолжения) цикла.

Для представленного фрагмента программы это можно сделать, например, так, как показано на рис. 2.13, а управляющий граф представить в виде рис. 2.14.

Номера строк	Строки программы
1	N = 5;
2	S = 0;
3	i = 1;
4	one: S += i;
5	i=i+1;
6	if (i<= N) goto one;
7	Console.WriteLine("s = {0:d}", S);

Рис. 2.13. Модифицированный фрагмент программы вычисления суммы членов натурального ряда

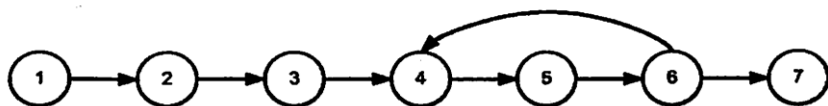


Рис. 2.14. Управляющий граф программы с циклом без объединения вершин

Заметим, что в графе вершины с первой по третью соответствуют действиям, предшествующим циклу, вершины три и четыре – группе операторов, составляющих тело цикла (множеству повторяемых действий), а вершина шесть – действию, определяющему необходимость продолжения или прекращения выполнения цикла. Расчет цикломатического числа Маккейба даст следующий результат:

$$Z = m - n + 2 = 5 - 5 + 2 = 2,$$

где количество дуг в графе равно $m = 5$, количество вершин $n = 5$.

Рассмотрим, как строится граф управления для программы с циклом *while*. Вновь обратимся к вычислению суммы первых членов натурального ряда, т. е. к задаче предыдущего примера. Фрагмент программной реализации задачи представлена на рис. 2.15.

Номера строк	Строки программы
1	N = 5;
2	S = 0;
3	i = 1;
4	while (i <= N)
5	{
6	i ++
7	S + = i;
8	}
9	Console.WriteLine(" {0:d} ", S);

Рис. 2.15. Фрагмент программы вычисления суммы членов натурального ряда с применением оператора *while*

Так же как и в предыдущем случае с циклом *do*, заменим оператор цикла эквивалентными операторами.

На рис. 2.16 приведены номера операторов до их объединения и соответствующие им номера после процедуры их приведения к одной вершине. Как уже объяснялось выше, объединять несколько операторов в группу и представить ее одной вершиной можно в том случае, если это последовательность линейных (следующих один за другим) операторов без ветвления процедуры выполнения программы. Управляющий граф такого фрагмента приведен на рис. 2.17.

В этом графе управления количество дуг равно $m = 5$, количество вершин $n = 5$, цикломатическое число Маккейба

$$Z = m - n + 2 = 5 - 5 + 2 = 2.$$

Номера строк до объединения	Номера строк после объединения	Строки программы
1	1	N = 5;
2	1	S = 0;
3	1	i = 1;
4	2	one: if (i >= N) goto two
5	3	{
6	3	i ++
7	4	S + = i;
8	5	goto one
		}
		two: Console.WriteLine("{0:d} ", S);

Рис. 2.16. Фрагмент программы вычисления суммы членов натурального ряда с применением оператора *while*

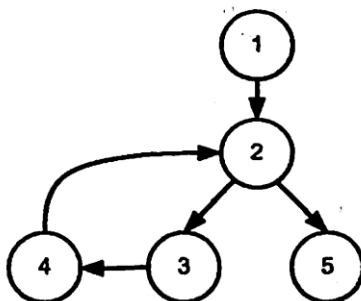


Рис. 2.17. Граф управления программой с циклом *while*

Как и для случая циклического оператора *do*, для отладки программы с оператором *while* может потребоваться два теста.

Теперь рассмотрим особенности построения графа управления для программы с циклом *for*. При этом предварительно вспомним, что циклы могут иметь две реализации: с предусловием и постусловием. Два описанных далее способа реализации цикла таковыми и являются. Оператор *do* реализуется с постусловием. В случае если количество повторений цикла задается таким, что цикл не должен выполняться ни разу, такой цикл даст неверный результат. Так, для рассмотренного ранее примера суммирования нескольких чисел натурального ряда при $N = 0$ результат (сумма) должна быть равна 0, в то время как программа сформирует результат 1. Случай же с предусловием (цикл типа *while*) даст ожидаемый результат, равный 0.

Еще одна особенность, которую следует отметить, состоит в том, что циклическая структура обязательно содержит три элемента: действия до начала цикла, тело цикла – повторяющиеся действия и условия окончания (или продолжения) цикла. Первый из элементов всегда выполняется первым. Для циклов с предусловием вторым выполняется проверка, а затем тело цикла. Циклы с постусловием реализованы по-другому: сначала выполняется тело цикла, а затем осуществляется проверка. Если говорить о цикле *for*, то он может быть реализован как по схеме с предусловием, так и с постусловием. Для используемых в данном пособии средств реализации языков C, C# и C++ оператор *for* реализован по схеме с предусловием и его граф управления программой похож на аналогичную структуру для оператора цикла *while*.

В качестве примера рассмотрим, как построить граф управления программой, вычисляющей число Фибоначчи по заданному номеру. Напомним, что эти числа вычисляются (для неотрицательных целых чисел) по следующему соотношению:

$$f_0 = 0; f_1 = 1; \dots; f_i = f_{i-1} + f_{i-2}; i \geq 2.$$

Текст программы, решающей поставленную задачу на языке программирования C, может быть таким, как показано на рис. 2.18.

Номера строк	Строки программы
1	<code>n=7;</code>
2	<code>f0=0;</code>
3	<code>f1=1;</code>
4	<code>for(i=2; i<=n; i++)</code>
5	<code>{</code>
6	<code> f2=f0+f1;</code>
7	<code> f0=f1;</code>
8	<code> f1=f2</code>
9	<code>}</code>
10	<code>printf(" %d ", f2);</code>

Рис. 2.18. Фрагмент программы вычисления чисел Фибоначчи

Для построения графа потока управления надо преобразовать программу, заменив цикл. Приведем фрагмент этой программы в преобразованном виде, где операторы цикла заменены операторами прямого вычисления (рис. 2.19).

Номера строк до объединения	Номера строк после объединения	Строки программы
1	1	n=7;
2	1	a0=0;
3	1	a1=1;
4	1	i=2;
5	2	one: if (i>N) goto two
6	3	a2=a0+a1;
7	3	a0=a1;
8	3	a1=a2; ...
9	4	goto one
10	5	two: printf(" %d %d", (i-1), a2);

Рис. 2.19. Фрагмент программы вычисления чисел Фибоначчи после преобразования операторов цикла

Граф потока управления для такой программы приведен на рис. 2.20. Как и в предыдущем примере, в этом графе объединены линейные группы операторов.

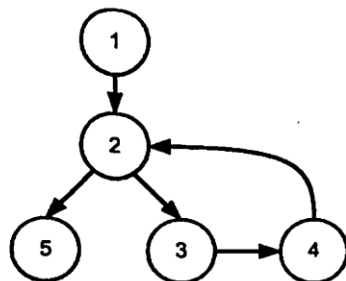


Рис. 2.20. Граф управления программой с циклом *for*

Рассмотрим более сложную задачу. Необходимо вычислить сумму факториалов заданного набора членов натурального ряда:

$$S = \sum_{i=1}^n i!$$

Для реализации решения задачи будем использовать цикл *for*.

Фрагмент программы решения этой задачи на языке C# может быть таким, как показано на рис. 2.21. Следует обратить внимание, что в этой программе есть вложенный цикл.

Номера строк	Строки программы
1	N = 5;
2	S = 0;
3	for (i=1; i<=N; i++)
4	{ f=1;
5	for (l=1; l<=i; l++) f=f*l;
6	S += i;
7	}
8	Console.WriteLine("{0:d} ", S);

Рис. 2.21. Фрагмент программы вычисления суммы факториалов

Выполним, как и в предыдущем примере, замену операторов цикла и получим, например, такой текст программы (рис. 2.22). Напомним, что мы говорим о возможных реализациях, поскольку по одному алгоритму можно разработать различные программы.

Номера строк до объединения	Номера строк после объединения	Строки программы
1	1	N = 5;
2	1	S = 0;
3	1	i=1;
4	2	one1: if (i>N) goto four;
5	3	f=1;
6	3	l=1;
7	4	two2: if (l>i) goto three;
8	5	f=f*l;
9	5	l++;
10	6	if (l<=i) goto two2;
11	7	three: S += f;
12	7	i ++;
13	8	if (i<=N) goto one1;
14	9	four: Console.WriteLine("{0:d} ", S);

Рис. 2.22. Фрагмент программы вычисления после замены операторов цикла

Теперь осуществим объединение линейных участков программы, которое приведет к тому тексту программы, на основе которого строится граф потока управления (рис. 2.23).

Граф потока управления для такой программы приведен на рис. 2.24, где группа вершин в тонированном контуре с пунктирными границами соответствуют вложенному циклу.

Номера строк	Строки программы
1	<code>N = 4; S = 0; i = 1;</code>
2	<code>one: if (i>N) goto four;</code>
3	<code>f=i; j=1;</code>
4	<code>two: if (l>i) goto three;</code>
5	<code>f = f*j; j++;</code>
6	<code>goto two;</code>
7	<code>three: S=S+f; i=i+1;</code>
8	<code>goto one;</code>
9	<code>four: Console.WriteLine("{0:d} ", S);</code>

Рис. 2.23. Фрагмент программы вычисления суммы факториалов

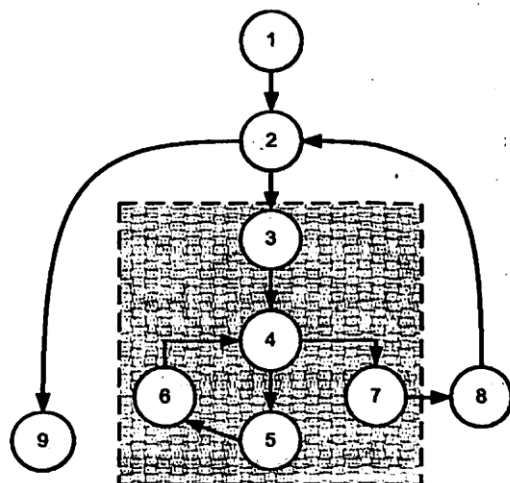


Рис. 2.24. Граф управления программой вычисления суммы факториалов

Цикломатическое число Маккейба для данного графа равно:

$$Z = m - n + 2 = 9 - 8 + 2 = 3.$$

Учитывая, что граф управления программой с циклом *for* может быть реализован также по схеме с постусловием, приведем пример построения графа управления и для этого случая. Заметим, что он будет похож на рассмотренный ранее граф с оператором *do*. По-прежнему рассматриваем задачу вычисления суммы факториалов. Произведем замену операторов цикла, тогда текст программы будет таким, как показано на рис. 2.25.

Номера строк до объединения	Номера строк после объединения	Строки программы
1	1	N = 5;
2	1	S = 0;
3	1	i=1;
4	2	one1: f=1;
5	3	l=1;
6	4	two2: f=f*l;
7	4	l++;
8	5	if (l<=i) goto two2;
9	6	S += f;
10	6	i ++;
11	7	if (i<=n) goto one1;
12	8	Console.WriteLine("{0:d} ", S);

Рис. 2.25. Фрагмент программы вычисления для случая с постусловием

Управляющий граф для варианта использования схемы с постусловием приведен на рис. 2.26.

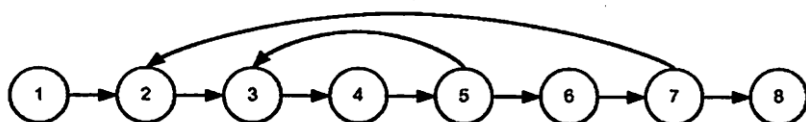


Рис. 2.26. Управляющий граф для схемы с постусловием

2.2. Задача «Расчет значений функции»

Вычислить значение функции $G = F(x, y)$, если

$$G = \begin{cases} \text{true}, & \text{если точка с координатами } (x, y) \text{ попадает в фигуру;} \\ \text{false}, & \text{если точка с координатами } (x, y) \text{ не попадает в фигуру.} \end{cases}$$

Фигура представляет собой сектор круга радиусом $R = 2$ в диапазоне углов $270^\circ \leq \varphi \leq 45^\circ$.

Разработать программу. В соответствии с разработанной программой составить блок-схему алгоритма решения задачи. На основании блок-схемы составить управляющий граф и оценить алгоритмическую сложность программы с использованием метрики Маккейба.

Реализация решения

Текст программы для реализации возможного решения поставленной задачи, разработанной с использованием языка программирования C#, приведен на рис. 2.27.

Номера строк	Строки программы
1	using System;
2	class Operator
3	{
4	public static void Main()
5	{
6	const double R = 2.0; .
7	double x, y; bool g;
8	char rep;
9	string str;
10	do
11	{
12	Console.Clear();
13	Console.Write("Введите X: ");
14	str = Console.ReadLine();
15	x = double.Parse(str);
16	Console.Write("Введите Y: ");
17	str = Console.ReadLine();
18	y = double.Parse(str);
19	g = false;
20	if (x * x + y * y <= R * R)
21	if (x >= 0)
22	if (y <= x)
23	g = true;
24	str = string.Format("G({0:f3},{1:f3}) = {2}", x, y, g);
25	Console.WriteLine(str);
26	Console.Write("Для повтора вычислений нажмите клавишу Y: ");
27	rep = char.Parse(Console.ReadLine());
28	Console.WriteLine();
29	} while (rep == 'Y' rep == 'y');
30	}
31	}

Рис. 2.27. Программа расчета значения функции

Алгоритм решения задачи выглядит так, как показано на рис. 2.28.

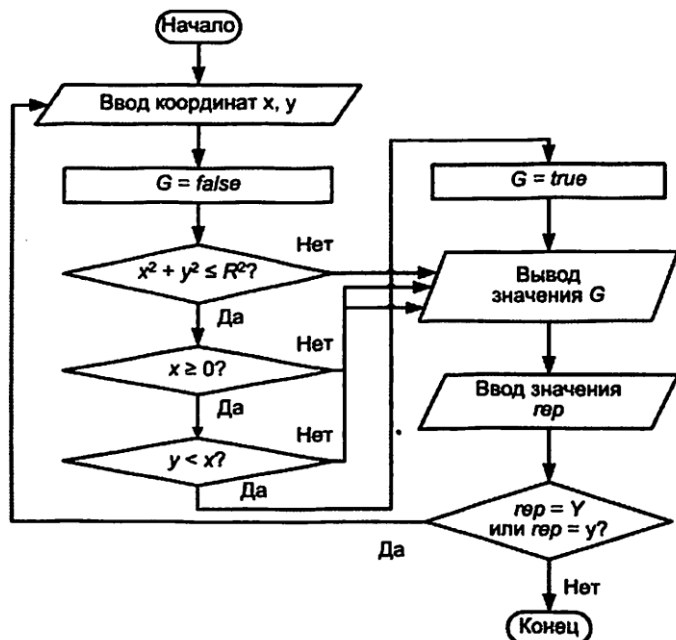


Рис. 2.28. Алгоритм вычисления значений функции

Оценка алгоритмической сложности

Граф потока управления для задачи расчета значений функции приведен на рис. 2.29. Тонированные вершины обозначают операторы ветвления.

Проведем оценку алгоритмической сложности графа по первому критерию. Определим минимальный набор маршрутов, проходящих через каждый оператор ветвления и по каждой дуге.

Для составленного управляющего графа можно выделить два маршрута:

$$m1: 1-2-\underline{3}-\underline{4}-\underline{5}-6-7-8-\underline{9}-2-\underline{3}-7-8-\underline{9}-10; \quad p_1 = 6;$$

$$m2: 1-2-\underline{3}-\underline{4}-7-8-\underline{9}-2-\underline{3}-\underline{4}-\underline{5}-7-8-\underline{9}-10; \quad p_2 = 7.$$

Данные маршруты проходят по всем вершинам ветвления, причем хотя бы один раз по каждой дуге графа, представленного на рис. 2.29, что соответствует требованиям первого критерия.

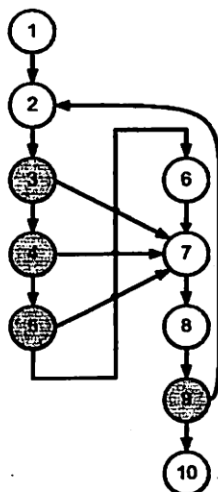


Рис. 2.29. Граф управления программой расчета значений функции

В перечне участков маршрутов номера вершин ветвления выделены полужирным шрифтом с подчеркиванием. Таким образом, в соответствии с первым критерием оценки алгоритмической сложности необходимое количество маршрутов равно 2, количество вершин ветвления в маршрутах определяет уровень сложности:

$$S_1 = p_1 + p_2 = 6 + 7 = 13.$$

Проведем оценку алгоритмической сложности по второму критерию. Необходимо определить число проверок каждого линейно независимого цикла и линейно независимого ациклического участка программы. Количество проверок определяется цикломатическим числом графа, которое определяется следующим соотношением:

$$Z = n_b + 1,$$

где n_b — число вершин ветвления.

Число вершин ветвления в составленном графе составляет 4, отсюда

$$Z = n_b + 1 = 4 + 1 = 5.$$

Таким образом, общее число циклических и ациклических участков равно 5. Выделим маршруты на заданном графе:

Для выделения маршрутов можно использовать матрицу достижимости, которая представляет собой для полученного графа управления квадратную таблицу размером 10×10 . Номер столбца матрицы определяет номер вершины графа, из которого можно достичь другие вершины этого же графа, используя циклические и ациклические маршруты. Номера строк определяют номера достижимых вершин графа управления. Из вершины 1 возможно достичь вершины со второй по десятую, т. е. достижимы все вершины графа (см. рис. 2.29). Для первого столбца матрицы достижимости строки, начиная со второй, заполняются единицами (табл. 2.4). Далее аналогично заполняются столбцы для остальных вершин.

Таблица 2.4. Матрица достижимости

	1	2	3	4	5	6	7	8	9	10
1										
2	1		1	1	1	1	1	1	1	
3	1	1		1	1	1	1	1	1	
4	1	1	1		1	1	1	1	1	
5	1	1	1	1		1	1	1	1	
6	1	1	1	1	1		1	1	1	
7	1	1	1	1	1	1		1	1	
8	1	1	1	1	1	1	1		1	
9	1	1	1	1	1	1	1	1		
10	1	1	1	1	1	1	1	1	1	

Выделенные диагональные элементы матрицы определяют номера вершин, которые входят в состав циклических маршрутов. Идентичные строки матрицы определяют номера вершин, которые входят в состав ациклических маршрутов.

Элементы, выделенные черным цветом, соответствуют маршруту m_5 : 2-3-4-5-6-7-8-9. Все строки матрицы получились идентичными, так как все вершины графа входят в состав ациклических маршрутов.

Проведем оценку алгоритмической сложности по третьему критерию. В соответствии с третьим критерием необходимо выделить все реально возможные маршруты управления:

$$m_1: 1-2-3-7-8-9-10; p_1 = 2;$$

$$m_2: 1-2-3-4-7-8-9-10; p_2 = 3;$$

$$m_3: 1-2-3-4-5-7-8-9-10; p_3 = 4;$$

$$m_4: 1-2-3-4-5-6-7-8-9-10; p_4 = 4;$$

$m_5: 1-2-\underline{3}-4-\underline{5}-6-7-8-\underline{9}-2-\underline{3}-4-\underline{5}-6-7-8-\underline{9}-10; p_5 = 8;$

$m_6: 1-2-\underline{3}-7-8-\underline{9}-2-\underline{3}-4-\underline{5}-6-7-8-\underline{9}-10; p_6 = 6;$

$m_7: 1-2-\underline{3}-4-7-8-\underline{9}-2-\underline{3}-4-\underline{5}-6-7-8-\underline{9}-10; p_7 = 7;$

$m_8: 1-2-\underline{3}-4-\underline{5}-7-8-\underline{9}-2-\underline{3}-4-\underline{5}-6-7-8-\underline{9}-10; p_8 = 8.$

Оценку структурной сложности программы рассчитываем по следующему соотношению:

$$S_3 = p_1 + p_2 + p_3 + p_4 + p_5 + p_6 + p_7 + p_8 = 2 + 3 + 4 + 4 + 8 + 6 + 7 + 8 = 42.$$

Исходя из полученных результатов расчета метрик структурной сложности по первому ($S_1 = 8$), второму ($S_2 = 17$) и третьему ($S_3 = 42$) критериям выделения маршрутов можно сделать вывод, что программа, характеризуемая заданным графом управления, имеет невысокую алгоритмическую сложность, так как количество используемых в тексте операторов условий 5, для проверки которых необходимо проверить от 5 до 42 тестовых вариантов исходных данных.

Оценим алгоритмическую сложность программы на основе метрики Маккейба. В соответствии с теорией Маккейба сложность алгоритма оценивается величиной цикломатического числа, которая определяется по следующему соотношению: $Z = m - n + 2$, где m – количество дуг управляющего графа, построенного на основе алгоритма программы, n – количество вершин графа. В соответствии с полученным управляющим графом $m = 13$, $n = 10$, тогда цикломатическое число равно:

$$Z = m - n + 2 = 13 - 10 + 2 = 5.$$

Таким образом, в соответствии со значением цикломатического числа ($Z = 5$) в полученном графе управления программой можно выделить пять независимых контуров, которые определяют пять управляющих маршрутов, ведущих из начальной вершины в конечную. Значение цикломатического числа для полученного графа ($Z = 5$) не превышает значения 10, что говорит о незначительной сложности алгоритма решения задачи расчета значений функции.

2.3. Задача «Замена строк матрицы»

Дана таблица вещественных чисел размером $N \times M$ элементов. Размер таблицы вводится с клавиатуры. Поменять местами строки таблицы по правилу: строка с номером 0 меняется с последней, строка с номером 1 – с предпоследней и т.д.

Так, например, пусть в исходном состоянии задана таблица

1,0	2,0	3,0	4,0
5,0	6,0	7,0	8,0
9,0	10,0	11,0	12,0

тогда в преобразованном состоянии мы должны получить

9,0	10,0	11,0	12,0
5,0	6,0	7,0	8,0
1,0	2,0	3,0	4,0

...

Разработать программу. В соответствии с разработанной программой составить блок-схему алгоритма решения задачи. На основании блок-схемы составить управляющий граф программы и оценить ее алгоритмическую сложность.

Реализация решения

Текст программы для реализации возможного решения поставленной задачи, разработанной с использованием языка программирования C#, приведен на рис. 2.30.

Номера строк	Строки программы
1	namespace Ex
2	{
3	class Перестановка
4	{
5	static void Main()
6	{
7	int n, m, mp, ном1, ном2, i, j;
8	double[,] a;
9	double b;
10	ConsoleKeyInfo клавиша;
11	do
12	{
13	Console.Clear();
14	Console.Write("Сколько строк: ");
15	n = int.Parse(Console.ReadLine());
16	Console.Write("Сколько столбцов: ");
17	m = int.Parse(Console.ReadLine());
18	a = new double[n, m];
19	for (i = 0; i < n; i++)
20	for (j = 0; j < m; j++)

```

21 {
22 Console.WriteLine("Элемент[{0},{1}]: ", i, j);
23 a[i, j] = double.Parse(Console.ReadLine());
24 }
25 Console.WriteLine("\nИсходная таблица");
26 for (i = 0; i < n; i++, Console.WriteLine())
27 for (j = 0; j < m; j++)
28 Console.Write(" {0,6:f2}", a[i, j]);
29 mp = n/2;
30 for(ном1=0,ном2=n-1; ном1<mp; ном1++,ном2--)
31 for (j = 0; j < m; j++)
32 {
33 b = a[ном1, j];
34 a[ном1, j] = a[ном2, j];
35 a[ном2, j] = b;
36 }
37 Console.WriteLine("\nТаблица после перестановки строк");
38 for (i = 0; i < n; i++, Console.WriteLine())
39 for (j = 0; j < m; j++)
40 Console.Write(" {0,6:f2}", a[i, j]);
41 Console.WriteLine("\nДля выхода нажмите клавишу ESC");
42 клавиша = Console.ReadKey(true);
43 } while (клавиша.Key!= ConsoleKey.Escape);
44 }
45 }
46 }

```

Рис. 2.30. Текст программы «Замена строк матрицы»

Алгоритм решения задачи выглядит так, как показано на рис. 2.31.

Оценка алгоритмической сложности

Граф потока управления приведен на рис. 2.32. Тонированные вершины обозначают операторы ветвления.

Проведем оценку алгоритмической сложности графа по первому критерию. Определим минимальный набор маршрутов, проходящих через каждый оператор ветвления и по каждой дуге.

Для составленного графа можно выделить один маршрут:

$m_1: 1-2-3-4-5-4-6-3-7-8-9-10-9-11-8-12-13-14-15-14-16-13-17-18-19-20-19-21-18-22-2-3-7-8-12-13-17-18-22-23;$
 $p_1 = 22.$

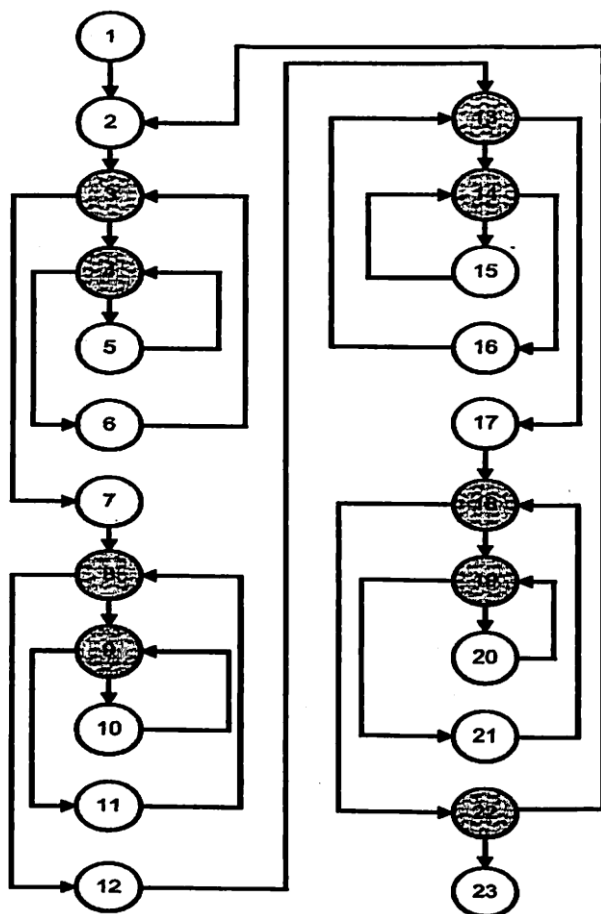


Рис. 2.32. Граф потока управления программой замены строк матрицы

Необходимо отметить, что уровень сложности данной программы достаточно высокий.

Проведем оценку алгоритмической сложности по второму критерию. Необходимо определить число проверок каждого линейно независимого цикла и линейно независимого ациклического участка программы. Количество проверок определяется цикломатическим числом графа. Число вершин ветвления в составленном графе составляет 9, отсюда $Z = n_v + 1 = 9 + 1 = 10$.

Таким образом, общее число циклических и ациклических участков составляет 10. Выделим маршруты на полученном графе:

- ациклические маршруты:

$m1: 1-2-3-4-5-4-6-3-7-8-9-10-9-11-8-12-13-14-15-14-16-13-17-18-19-20-19-21-18-22-23; p_1 = 17;$

- циклические маршруты:

$m2: 4-5; p_2 = 1;$

$m3: 3-4-5-4-6; p_3 = 3;$

$m4: 9-10; p_4 = 1;$

$m5: 8-9-10-9-11; p_5 = 3;$

$m6: 14-15; p_6 = 1;$

$m7: 13-14-15-14-16; p_7 = 3;$

$m8: 19-20; p_8 = 1;$

$m9: 18-19-20-19-21; p_9 = 3;$

$m10: 2-3-7-8-12-13-17-18-22; p_{10} = 5.$

Тестирование программы по указанным маршрутам позволит проверить все операторы ветвления программы. Метрика структурной сложности определяется по следующему соотношению:

$$S_2 = p_1 + p_2 + p_3 + p_4 + p_5 + p_6 + p_7 + p_8 + p_9 + p_{10} = 17 + 1 + 3 + 1 + 3 + 1 + 3 + 1 + 3 + 5 = 38.$$

Для организации автоматического анализа заданного графа по второму критерию с помощью вычислительных средств построим матрицы смежности и достижимости.

Напомним, что матрица смежности представляет собой квадратную матрицу, размер которой определяется количеством вершин графа. Полученный граф имеет 23 вершины, следовательно, матрица смежности будет иметь размер 23×23 . При заполнении матрицы следует придерживаться следующего правила: для дуги, соединяющей вершину 1 с вершиной 2 (см. рис. 2.32), в первый столбец и вторую строку матрицы смежности записываем 1 (табл. 2.5). Номер столбца соответствует номеру вершины, из которой выходит дуга, номер строки соответствует номеру вершины, в которую входит дуга. Таким образом заполняем матрицу для всех дуг управляющего графа.

Для выделения маршрутов можно использовать матрицу достижимости, которая представляет собой для полученного графа управления квадратную таблицу размером 23×23 . Номер столбца матрицы определяет номер вершины графа, из которого возможно достижение других вершин этого же графа, используя циклические и ациклические маршруты.

Таблица 2.5. Матрица смежности

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
1																							
2	1																					1	
3		1				1																	
4			1		1																		
5				1																			
6				1																			
7			1																				
8							1				1												
9								1		1													
10									1														
11									1														
12							1																
13											1					1							
14												1		1									
15													1										
16													1										
17												1											
18																	1				1		
19																		1		1			
20																			1				
21																			1				
22																		1					
23																						1	

Номера строк определяют номера достижимых вершин графа управления. Из вершины 1 возможно достичь вершины со 2-й по 23-ю, т. е. достижимы все вершины графа (см. рис. 2.32). Для первого столбца матрицы достижимости строки, начиная со второй, заполняются единицами (табл. 2.6). Далее заполняются столбцы для остальных вершин аналогично. Выделенные диагональные элементы матрицы определяют номера вершин, которые входят в состав циклических маршрутов. Идентичные строки матрицы определяют номера вершин, которые входят в состав ациклических маршрутов.

Диагональные элементы, выделенные курсивом для различных вариантов шрифтов, соответствуют m_{10} : 2-3-7-8-12-13-17-18-22. Диагональные элементы, выделенные курсивом с подчеркиванием, соответствуют m_3 : 3-4-5-4-6. На указанном участке элементы, выделенные черной заливкой, соответствуют m_2 : 4-5. Выделенные элементы полужирным курсивом соответствуют m_5 : 8-9-10-9-11.

Таблица 2.6. Матрица достижимости

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
1																							
2	1	<i>1</i>	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
3	1	1	<i>1</i>	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
4	1	1	1	<i>1</i>	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
5	1	1	1	1	<i>1</i>	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
6	1	1	1	1	1	<i>1</i>	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
7	1	1	1	1	1	1	<i>1</i>	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
8	1	1	1	1	1	1	1	<i>1</i>	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
9	1	1	1	1	1	1	1	1	<i>1</i>	1	1	1	1	1	1	1	1	1	1	1	1	1	1
10	1	1	1	1	1	1	1	1	1	<i>1</i>	1	1	1	1	1	1	1	1	1	1	1	1	1
11	1	1	1	1	1	1	1	1	1	1	<i>1</i>	1	1	1	1	1	1	1	1	1	1	1	1
12	1	1	1	1	1	1	1	1	1	1	1	<i>1</i>	1	1	1	1	1	1	1	1	1	1	1
13	1	1	1	1	1	1	1	1	1	1	1	1	<i>1</i>	1	1	1	1	1	1	1	1	1	1
14	1	1	1	1	1	1	1	1	1	1	1	1	1	<i>1</i>	1	1	1	1	1	1	1	1	1
15	1	1	1	1	1	1	1	1	1	1	1	1	1	1	<i>1</i>	1	1	1	1	1	1	1	1
16	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	<i>1</i>	1	1	1	1	1	1	1
17	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	<i>1</i>	1	1	1	1	1	1
18	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	<i>1</i>	1	1	1	1	1
19	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	<i>1</i>	1	1	1	1
20	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	<i>1</i>	1	1	1
21	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	<i>1</i>	1	1
22	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	<i>1</i>	1
23	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	<i>1</i>

На указанном участке элементы, выделенные темной заливкой, соответствуют m_4 : 2–10. Диагональные элементы, выделенные полужирным курсивом с подчеркиванием, соответствуют m_7 : 13–14–15–14–16. Залитые ячейки темным цветом на указанном участке соответствуют m_6 : 14–15. Диагональные элементы, выделенные курсивом иного шрифта, соответствуют m_9 : 18–19–20–19–21. Ячейки на указанном участке, залитые темным цветом, соответствуют m_8 : 19–20. Все строки матрицы достижимости идентичны и включают все номера вершин ациклического маршрута.

Проведем оценку алгоритмической сложности по третьему критерию. В соответствии с третьим критерием необходимо выделить все реально возможные маршруты управления:

m_1 : 1–2–3–4–5–4–6–3–7–8–9–10–9–11–8–12–13–14–15–14–16–
–13–17–18–19–20–19–21–18–22–23; $p_1=16$;

m_2 : 1–2–3–7–8–12–13–17–18–22–2–3–4–6–3–7–8–9–11–8–12–13–
–14–16–13–17–18–19–21–18–22–23; $p_2=18$;

m_3 : 1-2-3-4-5-4-6-3-7-8-9-10-9-11-8-12-13-14-15-14-16-
 -13-17-18-19-20-19-21-18-22-2-3-4-6-3-7-8-9-11-8-12-
 -13-14-16-13-17-18-19-21-18-22-23; $p_3=29$.

Оценку структурной сложности программы проводим по следующему соотношению:

$$S_3 = p_1 + p_2 + p_3 = 16 + 18 + 29 = 63.$$

Исходя из полученных результатов расчета метрик структурной сложности по первому ($S_1 = 22$), второму ($S_2 = 38$) и третьему ($S_3 = 63$) критериям выделения маршрутов можно сделать вывод, что программа, характеризуемая представленным графом управления, имеет высокую алгоритмическую сложность, так как количество используемых в тексте операторов условий 9, для проверки которых необходимо провести от 22 до 63 тестовых вариантов исходных данных.

Оценим алгоритмическую сложность программы на основе метрики Маккейба. В соответствии с теорией Маккейба сложность алгоритма оценивается величиной цикломатического числа, которая определяется по следующему соотношению: $Z = m - n + 2$, где m – количество дуг управляющего графа, построенного на основе алгоритма программы, n – количество вершин графа. В соответствии с управляющим графом $m = 30$ (см. рис. 2.32), $n = 23$, тогда цикломатическое число равно:

$$Z = m - n + 2 = 30 - 23 + 2 = 9.$$

Таким образом, в соответствии со значением цикломатического числа ($Z = 9$) в полученном графе управления программой можно выделить девять независимых контуров, которые определяют девять управляющих маршрутов, ведущих из начальной вершины в конечную. Значение цикломатического числа для полученного графа не превышает значения 10, но очень близко к нему, что говорит о высокой сложности разработанного алгоритма, которая близка к критической.

2.4. Задача «Объединение аргументов командной строки»

Из принятых аргументов командной строки сформировать новую строку путем объединения всех аргументов командной строки. Каждый аргумент командной строки сортируется посимвольно по убыва-

нию в машинном алфавите. В объединенной строке аргументы командной строки разделяются символом двоеточия:

- командная строка: ex1.exe 1234 89 67;
- объединенная строка: 4321:98:76.

Запуск исполняемого файла должен осуществляться через главное меню операционной системы (кнопка «Пуск» → «Выполнить»).

Разработать программу. В соответствии с разработанной программой составить блок-схему алгоритма решения задачи. На основании блок-схемы сформировать граф потока управления программой и оценить ее алгоритмическую сложность.

Реализация решения

Текст программы для реализации возможного решения поставленной задачи, разработанной с использованием языка программирования C#, приведен на рис. 2.33.

Номера строк	Строки программы
1	using System;
2	namespace EX1
3	{
4	class Program
5	{
6	static void Main(string[] args)
7	{
8	string union;
9	char[] word;
10	int i;
11	for (i = 0; i < args.Length; i++)
12	{
13	word = new char[args[i].Length];
14	args[i].CopyTo(0, word, 0, args[i].Length);
15	Array.Sort(word, 0, word.Length);
16	Array.Reverse(word, 0, word.Length);
17	args[i] = new string(word);
18	}
19	union = string.Join(":", args);
20	Console.WriteLine(union);
21	Console.ReadKey();
22	}
23	}
24	}

Рис. 2.33. Текст программы объединения аргументов командной строки

Алгоритм решения задачи приведен на рис. 2.34.

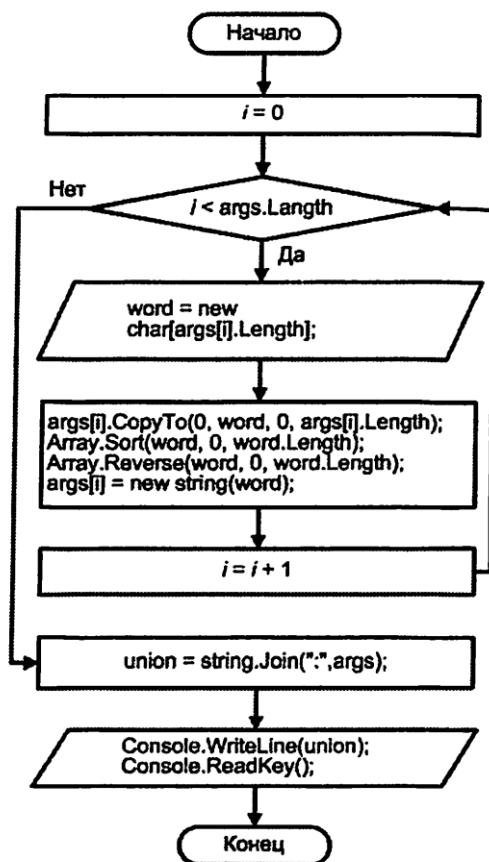


Рис. 2.34. Управляющий граф программы объединения аргументов

Оценка алгоритмической сложности

Граф потока управления приведен на рис. 2.35. Тонированная вершина обозначает оператор ветвления.

Проведем оценку алгоритмической сложности графа по первому критерию. Определим минимальный набор маршрутов, проходящих через каждый оператор ветвления и по каждой дуге.

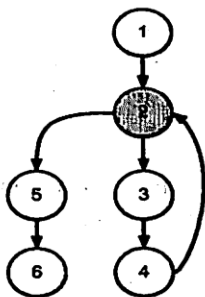


Рис. 2.35. Управляющий граф программы объединения аргументов

Для составленного графа можно выделить один маршрут:

$$m1: 1-\underline{2}-3-4-2-5-6; p_1 = 1.$$

Выделенный маршрут проходит по всем вершинам ветвления и хотя бы один раз по каждой дуге графа управления потоком рассматриваемой задачи, что соответствует требованиям первого критерия.

В перечне выделенного маршрута номер вершины ветвления выделен полужирным шрифтом с подчеркиванием. Количество маршрутов – 1, количество вершин ветвления в маршруте – 1.

Уровень сложности алгоритма по первому критерию выбора маршрутов равен:

$$S_1 = p_1 = 1.$$

Алгоритм данной программы весьма прост.

Проведем оценку алгоритмической сложности по второму критерию выбора маршрутов. Необходимо определить число проверок каждого линейно независимого цикла и линейно независимого ациклического участка программы. Количество проверок определяется цикломатическим числом графа, которое определяется следующим соотношением: $Z = n_b + 1$, где n_b – число вершин ветвления. Число вершин ветвления в составленном графе составляет 1, отсюда

$$Z = n_b + 1 = 1 + 1 = 2.$$

Таким образом, общее число циклических и ациклических участков составляет 2. Выделим маршруты на полученном графе:

- ациклический маршрут: $m1: 1-\underline{2}-3-4-2-5-6; p_1 = 1;$
- циклический маршрут: $m2: \underline{2}-3-4; p_2 = 1.$

Тестирование программы по указанным маршрутам позволит проверить все операторы ветвления программы. Метрика структурной сложности определяется по следующему соотношению:

$$S_2 = p_1 + p_2 = 1 + 1 = 2.$$

Для организации автоматического анализа заданного графа по второму критерию с помощью вычислительных средств построим матрицы смежности и достижимости.

Напомним, что матрица смежности представляет собой квадратную матрицу, размер которой определяется количеством вершин графа. Полученный граф имеет 6 вершин, следовательно, матрица смежности будет иметь размер 6×6 . При заполнении матрицы следует придерживаться следующего правила: для дуги, соединяющей вершину 1 с вершиной 2 (см. рис. 2.35), в первый столбец и вторую строку матрицы смежности записываем 1 (см. табл. 2.7). Номер столбца соответствует номеру вершины, из которой выходит дуга, номер строки соответствует номеру вершины, в которую входит дуга. Таким образом заполняем матрицу для всех дуг управляющего графа.

Таблица 2.7. Матрица смежности

	1	2	3	4	5	6
1						
2	1			1		
3		1				
4			1			
5		1				
6					1	

Для выделения маршрутов можно использовать матрицу достижимости, которая представляет собой для полученного графа управления квадратную таблицу размером 6×6 . Номер столбца матрицы определяет номер вершины графа, из которого возможно достичь другие вершины этого же графа, используя циклические и ациклические маршруты. Номера строк определяют номера достижимых вершин графа управления. Из вершины 1 возможно достичь вершины со 2-й по 6-ю, т. е. достижимы все вершины графа (см. рис. 2.35). Для первого столбца матрицы достижимости строки, начиная со второй, заполняются единицами (см. табл. 2.8). Далее заполняются столбцы для остальных вершин аналогичным образом. Выделенные диагональные элементы матрицы

определяют номера вершин, которые входят в состав циклических маршрутов. Идентичные строки матрицы определяют номера вершин, которые входят в состав ациклических маршрутов.

Таблица 2.8. Матрица достижимости

	1	2	3	4	5	6
1						
2	1	1	1	1		
3	1	1	1	1		...
4	1	1	1	1		
5	1	1	1	1		
6	1	1	1	1	1	

Диагональные элементы, выделенные темным цветом, соответствуют циклическому маршруту m_2 : 2–3–4. В шестой строке определены номера столбцов, соответствующие номерам вершин графа, входящих в состав ациклического маршрута m_1 : 1–2–3–4–2–5–6.

Сложность алгоритма по второму критерию отбора маршрутов невелика ($S_2 = 2$).

Проведем оценку алгоритмической сложности по третьему критерию, в соответствии с которым необходимо выделить все реально возможные маршруты управления:

$$m_1: 1-2-3-4-2-5-6; p_1=1;$$

$$m_2: 1-2-5-6; p_2=1.$$

Оценку структурной сложности программы рассчитываем по следующему соотношению:

$$S_3 = p_1 + p_2 = 1 + 1 = 2.$$

Исходя из полученных результатов расчета метрик структурной сложности по первому ($S_1 = 1$), второму ($S_2 = 2$) и третьему ($S_3 = 2$) критериям выделения маршрутов можно сделать вывод, что программа, характеризующая сформированным графом управления, имеет весьма низкую алгоритмическую сложность, так как используется всего один оператор условий, для проверки которых необходимо провести 2 тестовых варианта исходных данных.

Оценим алгоритмическую сложность программы на основе метрики Маккейба. В соответствии с теорией Маккейба сложность алгоритма оценивается величиной цикломатического числа, которая оп-

ределяется по следующему соотношению: $Z = m - n + 2$, где m – количество дуг управляющего графа, построенного на основе алгоритма программы, n – количество вершин графа. В соответствии с полученным управляющим графом $m = 6$, $n = 6$, тогда цикломатическое число равно:

$$Z = m - n + 2 = 6 - 6 + 2 = 2.$$

Таким образом, в соответствии со значением цикломатического числа ($Z = 2$) в полученном графе управления программой можно выделить два независимых контура, которые определяют два управляющих маршрута, ведущих из начальной вершины в конечную. Значение цикломатического числа для полученного графа весьма далеко от предельного значения 10, что говорит о низком уровне сложности разработанного алгоритма.

2.5. Задача «Проверка простого числа»

Программе с клавиатуры задается число. Необходимо разработать алгоритм для комплексного решения двух независимых задач:

- определить, является ли заданное число простым;
- определить количество цифр, составляющих заданное число.

По разработанному алгоритму построить граф потока управления и оценить сложность алгоритма программы по управляющему графу.

Реализация решения

Алгоритм решения задачи может выглядеть так, как показано на рис. 2.36, а граф потока управления приведен на рис. 2.37.

Тонированные вершины обозначают операторы ветвления.

Оценка алгоритмической сложности

Проведем оценку алгоритмической сложности графа по первому критерию. Определим минимальный набор маршрутов, проходящих через каждый оператор ветвления и по каждой дуге.

Первый маршрут:

$$m1: 1-2-3-4-5-6-4-5-6-10-11-2-3-7-8-9-7-8-9-10-11-12; p_1 = 8.$$

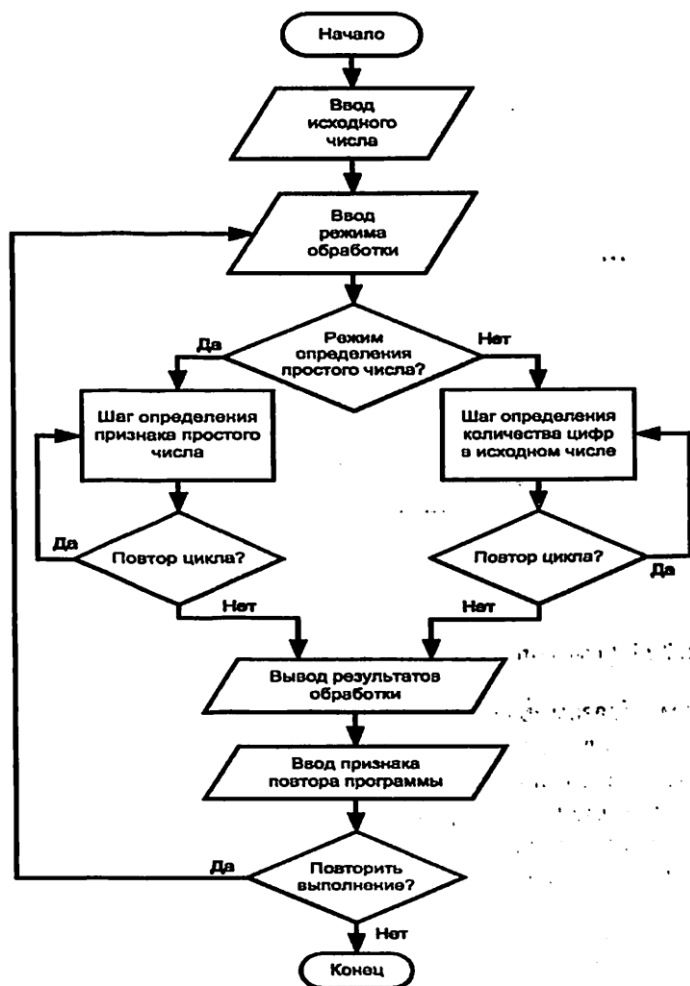


Рис. 2.36. Алгоритм решения задачи о простом числе

Данный маршрут проходит по всем вершинам ветвления и хотя бы один раз по каждой дуге графа, представленного на рис. 2.36), что соответствует требованиям первого критерия. В перечне участков маршрута номера вершин ветвления выделены полужирным шрифтом с подчеркиванием. Количество таких вершин в маршруте составляет 8.

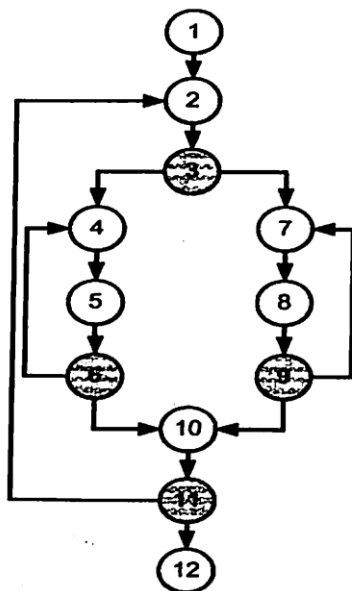


Рис. 2.37. Граф потока управления задачи определения простого числа

Таким образом, в соответствии с первым критерием оценки алгоритмической сложности количество маршрутов 1, количество вершин ветвления в маршруте 8, т. е. $S_1 = p_1 = 8$.

Проведем оценку алгоритмической сложности по второму критерию. При этом необходимо определить число проверок каждого линейно независимого цикла и линейно независимого ациклического участка программы. Количество проверок определяется цикломатическим числом графа, которое определяется следующим соотношением: $Z = n_b + 1$, где n_b – число вершин ветвления.

Число вершин ветвления в заданном графе составляет 4, откуда

$$Z = n_b + 1 = 4 + 1 = 5.$$

Таким образом, общее число циклических и ациклических участков составляет 5. Выделим эти маршруты в заданном графе:

- ациклические маршруты:

$$m1: 1-2-\underline{3}-4-5-\underline{6}-10-\underline{11}-12; p_1 = 3;$$

$$m2: 1-2-\underline{3}-7-8-\underline{9}-10-\underline{11}-12; p_2 = 3;$$

Для выделения маршрутов можно использовать матрицу достижимости, которая также представляет собой для заданного графа управления квадратную таблицу размером 12×12 .

Номер столбца матрицы определяет номер вершины графа, из которого возможно достичь другие вершины этого же графа, используя циклические и ациклические маршруты. Номера строк определяют номера достижимых вершин графа управления. Из вершины 1 возможно достичь вершины со 2-й по 12-ю, т. е. достижимы все вершины графа (см. рис. 2.37). Для первого столбца матрицы достижимости строки, начиная со второй, заполняются единицами (табл. 2.10). Далее заполняются столбцы для остальных вершин аналогично.

Таблица 2.10. Матрица достижимости для задачи простого числа

	1	2	3	4	5	6	7	8	9	10	11	12
1												
2	1	1	1	1	1	1	1	1	1	1	1	
3	1	1	1	1	1	1	1	1	1	1	1	
4	1	1	1	1	1	1	1	1	1	1	1	
5	1	1	1	1	1	1	1	1	1	1	1	
6	1	1	1	1	1	1	1	1	1	1	1	
7	1	1	1	1	1	1	1	1	1	1	1	
8	1	1	1	1	1	1	1	1	1	1	1	
9	1	1	1	1	1	1	1	1	1	1	1	
10	1	1	1	1	1	1	1	1	1	1	1	
11	1	1	1	1	1	1	1	1	1	1	1	
12	1	1	1	1	1	1	1	1	1	1	1	

Выделенные диагональные элементы матрицы определяют номера вершин, которые входят в состав циклических маршрутов. Идентичные строки матрицы определяют номера вершин, которые входят в состав ациклических маршрутов.

Элементы, выделенные черным цветом, соответствуют маршруту m_4 : 4–5–6, элементы, выделенные штриховкой, соответствуют маршруту m_5 : 7–8–9. Первые два «серых» элемента, «черные» элементы и последние два «серых» элемента соответствуют m_3 : 2–3–4–5–6–10–11.

Все строки матрицы получились идентичными, так как все вершины графа входят в состав ациклических маршрутов:

m_1 : 1–2–3–4–5–6–10–11–12;

m_2 : 1–2–3–7–8–9–10–11–12.

Проведем оценку алгоритмической сложности по третьему критерию. В соответствии с третьим критерием необходимо выделить все реально возможные маршруты управления:

- $m_1: 1-2-\underline{3}-4-5-\underline{6}-10-\underline{11}-12; p_1 = 3;$
 $m_2: 1-2-\underline{3}-7-8-\underline{9}-10-\underline{11}-12; p_2 = 3;$
 $m_3: 1-2-\underline{3}-4-5-\underline{6}-4-5-\underline{6}-10-\underline{11}-12; p_3 = 4;$
 $m_4: 1-2-\underline{3}-7-8-\underline{9}-7-8-\underline{9}-10-\underline{11}-12; p_4 = 4;$
 $m_5: 1-2-\underline{3}-4-5-\underline{6}-4-5-\underline{6}-10-\underline{11}-2-\underline{3}-4-5-\underline{6}-4-5-\underline{6}-10-\underline{11}-12; p_5 = 8;$
 $m_6: 1-2-\underline{3}-7-8-\underline{9}-7-8-\underline{9}-10-\underline{11}-2-\underline{3}-7-8-\underline{9}-7-8-\underline{9}-10-\underline{11}-12; p_6 = 8;$
 $m_7: 1-2-\underline{3}-4-5-\underline{6}-4-5-\underline{6}-10-\underline{11}-2-\underline{3}-7-8-\underline{9}-7-8-\underline{9}-10-\underline{11}-12; p_7 = 8;$
 $m_8: 1-2-\underline{3}-7-8-\underline{9}-7-8-\underline{9}-10-\underline{11}-2-\underline{3}-4-5-\underline{6}-4-5-\underline{6}-10-\underline{11}-12; p_8 = 8.$

Оценку структурной сложности программы рассчитываем по следующему соотношению:

$$S_3 = p_1 + p_2 + p_3 + p_4 + p_5 + p_6 + p_7 + p_8 = 3 + 3 + 4 + 4 + 8 + 8 + 8 + 8 = 46.$$

Исходя из полученных результатов расчета метрик структурной сложности по первому ($S_1 = 8$), второму ($S_2 = 11$) и третьему ($S_3 = 46$) критериям выделения маршрутов можно сделать вывод, что программа, характеризуемая заданным графом управления, имеет невысокую алгоритмическую сложность, так как количество используемых в тексте операторов условий равно 5.

Для тестирования программы с данным управляющим графом необходимо проверить от 8 до 46 тестовых маршрутов обработки исходных данных.

2.6. Задача «Сортировка массива»

Необходимо провести сортировку значений элементов одномерного массива по возрастанию.

По разработанному алгоритму построить управляющий граф и оценить алгоритмическую сложность возможной программы.

Реализация решения

Алгоритм решения задачи представлен на рис. 2.38.

В соответствии с разработанным алгоритмом граф потока управления будет иметь вид, показанный на рис. 2.39.

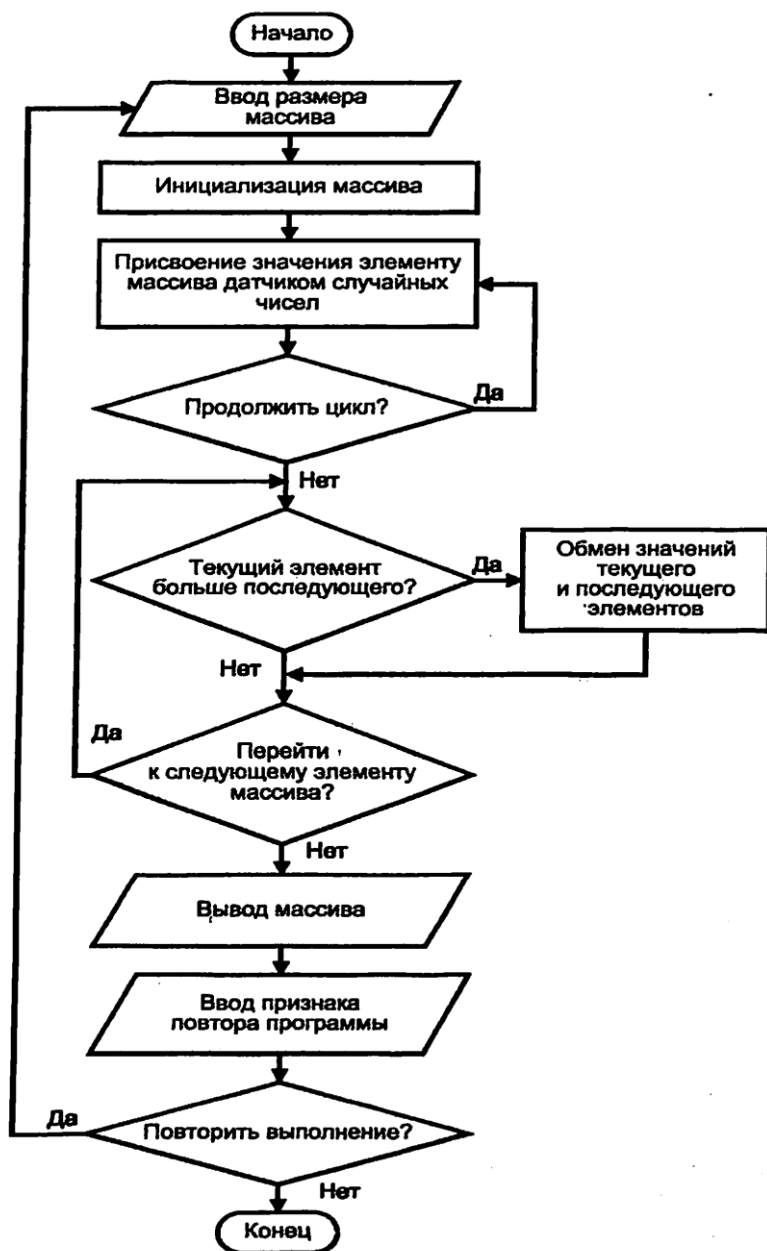


Рис. 2.38. Алгоритм решения задачи сортировки массива

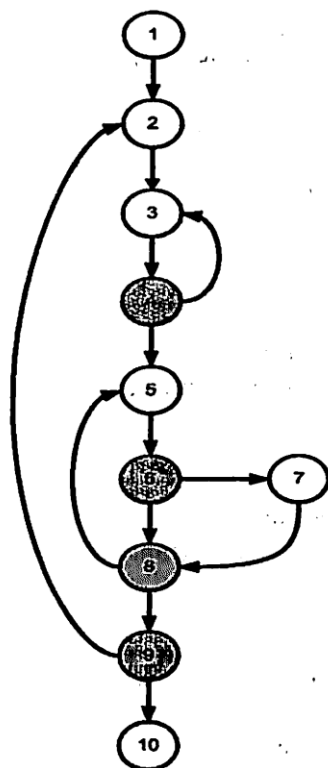


Рис. 2.39. Управляющий граф задачи сортировки массива

Оценка алгоритмической сложности

Проведем оценку алгоритмической сложности графа по первому критерию, для чего определим минимальный набор маршрутов, проходящих через каждый оператор ветвления и по каждой дуге.

Возможный маршрут управления:

$m_1: 1-2-3-\underline{4}-3-\underline{4}-5-\underline{6}-7-\underline{8}-5-\underline{6}-8-9-2-3-\underline{4}-5-\underline{6}-8-9-10; p_1 = 11.$

Анализ выделенного маршрута показывает, что он проходит по всем вершинам ветвления и хотя бы один раз по каждой дуге представленного графа. Минимальное количество маршрутов для заданного графа составляет 1, так как этим маршрутом охватываются все

дуги и вершины графа. Тонированные вершины обозначают операторы ветвления, в перечне пунктов маршрута номера этих вершин выделены шрифтом с подчеркиванием. Количество таких пунктов маршрута – 11. Таким образом, в соответствии с первым критерием алгоритмической сложности количество маршрутов 1, количество вершин ветвления в маршруте 11:

$$S_1 = p_1 = 11.$$

Проведем оценку алгоритмической сложности по второму критерию. Необходимо определить число проверок каждого линейно независимого цикла и линейно независимого ациклического участка программы. Количество проверок определяется цикломатическим числом графа, которое определяется следующим соотношением:

$$Z = n_b + 1,$$

где n_b – число вершин ветвления.

Число вершин ветвления в заданном графе составляет 4, следовательно,

$$Z = n_b + 1 = 4 + 1 = 5.$$

Таким образом, общее число циклических и ациклических маршрутов составляет 5. Выделим эти маршруты на заданном графе:

- ациклические маршруты:

$$m1: 1-2-3-\underline{4}-5-6-\underline{7}-8-\underline{9}-\underline{10}-11-\underline{12}-13; p_1 = 4;$$

$$m2: 1-2-3-\underline{4}-5-6-\underline{7}-\underline{9}-\underline{10}-11-\underline{12}-13; p_2 = 4;$$

- циклические маршруты:

$$m3: 3-\underline{4}; p_3 = 1;$$

$$m4: 5-\underline{6}-\underline{8}; p_4 = 2;$$

$$m5: 2-3-\underline{4}-5-\underline{6}-7-\underline{8}-\underline{9}; p_5 = 2.$$

Тестирование указанных маршрутов позволяет проверить все операторы ветвления программы. Метрика структурной сложности определяется по следующему соотношению:

$$S_2 = p_1 + p_2 + p_3 + p_4 + p_5 + p_6 = 4 + 4 + 1 + 2 + 4 = 15.$$

Для организации автоматического анализа заданного графа по второму критерию с помощью вычислительных средств построим матрицы смежности и достижимости.

Матрица смежности представляет собой квадратную матрицу, размер которой определяется количеством вершин графа. Заданный граф имеет 10 вершин, следовательно, матрица смежности будет иметь размер 10×10 . При заполнении матрицы следует придерживаться следующего правила: для дуги, соединяющей вершину 1 с вершиной 2 (см. рис. 2.39) в первый столбец и вторую строку матрицы смежности записываем 1 (табл. 2.11). ...

Таблица 2.11. Матрица смежности

	1	2	3	4	5	6	7	8	9	10
1										
2	1								1	
3		1		1						
4			1							
5				1				1		
6					1					
7						1				
8						1	1			
9								1		
10									1	

Номер столбца соответствует номеру вершины, из которой выходит дуга, номер строки соответствует номеру вершины, в которую входит дуга. Таким же образом заполняем матрицу для всех дуг управляющего графа.

Для выделения маршрутов можно использовать матрицу достижимости, которая представляет собой для заданного графа управления квадратную таблицу размером 10×10 . Номер столбца матрицы определяет номер вершины графа, из которого возможно достичь другие вершины этого же графа, используя циклические и ациклические маршруты. Номера строк определяют номера достижимых вершин графа управления. Из вершины с 1 возможно достичь вершины со 2-й по 10-ю, т. е. достижимы все вершины графа. Для первого столбца матрицы достижимости строки, начиная со второй, заполняются единицами (табл. 2.12). Далее заполняются столбцы для остальных вершин аналогично.

Выделенные диагональные элементы матрицы определяют номера столбцов-вершин, которые входят в состав циклических маршрутов. Идентичные строки матрицы определяют номера столбцов-вершин, которые входят в состав ациклических маршрутов.

Таблица 2.12. Матрица достижимости

	1	2	3	4	5	6	7	8	9	10
1										
2	1	1	1	1	1	1	1	1	1	1
3	1	1	1	1	1	1	1	1	1	1
4	1	1	1	1	1	1	1	1	1	1
5	1	1	1	1	1	1	1	1	1	1
6	1	1	1	1	1	1	1	1	1	1
7	1	1	1	1	1	1	1	1	1	1
8	1	1	1	1	1	1	1	1	1	1
9	1	1	1	1	1	1	1	1	1	1
10	1	1	1	1	1	1	1	1	1	1

Элементы, выделенные штриховкой в левую сторону, соответствуют маршруту m_3 , элементы, выделенные штриховкой в правую сторону, – маршруту m_4 . Элементы, помеченные единицами полужирного шрифта белого и черного цвета, соответствуют маршруту m_5 . Элементы, помеченные единицами белого цвета, соответствуют m_6 . Все строки матрицы получились идентичными, так как все вершины графа входят в состав ациклических маршрутов m_1 и m_2 .

Проведем оценку алгоритмической сложности по третьему критерию. В соответствии с третьим критерием необходимо выделить все реально возможные маршруты управления:

$$m_1: 1-2-3-4-5-6-7-8-9-10; p_1 = 4;$$

$$m_2: 1-2-3-4-5-6-8-9-10; p_2 = 4;$$

$$m_3: 1-2-3-4-3-4-5-6-7-8-5-6-8-9-10; p_3 = 6;$$

$$m_4: 1-2-3-4-5-6-8-9-2-3-4-5-6-8-9-10; p_4 = 8.$$

Оценку структурной сложности программы рассчитываем по следующему соотношению:

$$S_3 = p_1 + p_2 + p_3 + p_4 = 4 + 4 + 6 + 8 = 22.$$

Исходя из полученных результатов расчета метрик структурной сложности по первому ($S_1 = 11$), второму ($S_2 = 20$) и третьему ($S_3 = 41$) критериям выделения маршрутов можно сделать вывод, что программа, характеризуемая заданным графом управления, имеет невысокую алгоритмическую сложность, так как количество используемых в тексте операторов условий равно 4, для проверки которых необходимо провести от 11 до 22 тестовых вариантов исходных данных.

2.7. Задача «Поиск максимального числа»

Необходимо определить максимальное из натуральных чисел, не превышающих K , которое нацело делится на M . Числа K и M вводятся с клавиатуры по запросу программы. По разработанному алгоритму построить граф потока управления и оценить алгоритмическую сложность программы.

Реализация решения

Алгоритм решения задачи представлен на рис. 2.40.

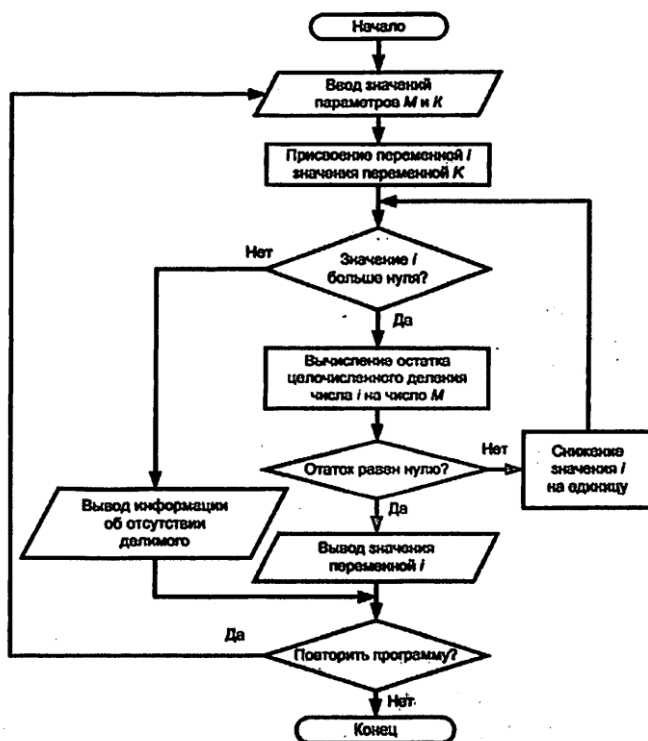


Рис. 2.40. Алгоритм решения задачи о поиске максимального числа

Граф потока управления для программы поиска максимального числа приведен на рис. 2.41.

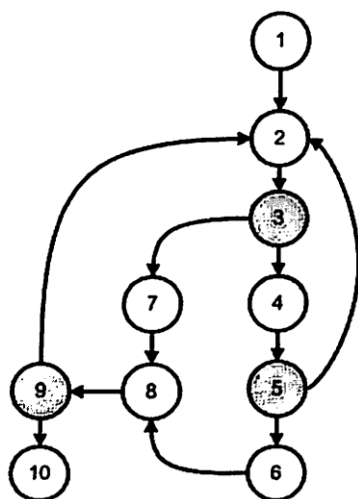


Рис. 2.41. Граф потока управления для задачи поиска максимального числа

Оценка алгоритмической сложности

Проведем оценку алгоритмической сложности графа по первому критерию. Определим минимальный набор маршрутов, проходящих через каждый оператор ветвления и по каждой дуге.

Возможный маршрут управления:

$$m1: 1-2-\underline{3}-4-\underline{5}-2-\underline{3}-4-5-6-8-\underline{2}-\underline{3}-7-8-\underline{2}-10; p_1 = 6.$$

Выделенный маршрут проходит по всем вершинам ветвления и хотя бы один раз по каждой дуге представленного графа. Минимальное количество маршрутов для заданного графа составляет 1, так как этим маршрутом охватываются все дуги и вершины графа. Тонировка вершин обозначает операторы ветвления. В перечне маршрута номера этих вершин выделены полужирным шрифтом с подчеркиванием. Если пересчитать полужирные номера выделенного маршрута, то их количество составляет 6. Таким образом, в соответствии с первым критерием алгоритмической сложности количество маршрутов 1, количество вершин ветвления в маршруте 6: $S_1 = p_1 = 6$.

Проведем оценку алгоритмической сложности по второму критерию. Необходимо определить число проверок каждого линейно независимого цикла и линейно независимого ациклического участка программы. Количество проверок определяется цикломатическим числом графа, которое определяется следующим соотношением:

$$Z = n_b + 1,$$

где n_b — число вершин ветвления.

Число вершин ветвления в заданном графе составляет 3, следовательно, цикломатическое число определяется следующим образом:

$$Z = n_b + 1 = 3 + 1 = 4.$$

Таким образом, общее число циклических и ациклических маршрутов составляет 4. Выделим эти маршруты на заданном графе:

- ациклические маршруты:

$$m1: 1-2-\underline{3}-4-\underline{5}-8-\underline{9}-10; p_1 = 3;$$

$$m2: 1-2-\underline{3}-7-8-\underline{9}-10; p_2 = 2;$$

- циклические маршруты:

$$m3: 2-\underline{3}-4-\underline{5}; p_3 = 2;$$

$$m4: 2-\underline{3}-7-8-\underline{9}; p_4 = 2.$$

Тестирование указанных маршрутов позволяет проверить все операторы ветвления программы. Метрика структурной сложности определяется по следующему соотношению:

$$S_2 = p_1 + p_2 + p_3 + p_4 = 3 + 5 + 2 + 2 = 12.$$

Для организации автоматического анализа заданного графа по второму критерию с помощью вычислительных средств построим матрицы смежности и достижимости.

Матрица смежности представляет собой квадратную матрицу, размер которой определяется количеством вершин графа. Заданный граф имеет 10 вершин, следовательно, матрица смежности будет иметь размер 10×10 . При заполнении матрицы следует придерживаться следующего правила: для дуги, соединяющей вершину 1 с вершиной 2, в первый столбец и вторую строку матрицы смежности записываем 1 (табл. 2.13). Номер столбца соответствует номеру вершины, из которой выходит дуга, номер строки соответствует номеру вершины, в которую входит дуга. Таким же образом заполняем матрицу для всех дуг управляющего графа.

Таблица 2.13. Матрица смежности для поиска максимального числа

	1	2	3	4	5	6	7	8	9	10
1										
2	1				1				1	
3		1								
4			1							
5				1						
6					1					
7			1							
8						1	1			
9								1		
10									1	

Для выделения маршрутов можно использовать матрицу достижимости, которая представляет собой для заданного графа управления квадратную таблицу размером 10×10 . Номер столбца матрицы определяет номер вершины графа, из которого можно достичь другие вершины этого же графа, используя циклические и ациклические маршруты. Номера строк определяют номера достижимых вершин графа управления. Из вершины с 1 возможно достичь вершины со 2-й по 10-ю, т. е. достижимы все вершины графа (рис. 2.41.). Для первого столбца матрицы достижимости строки, начиная со второй, заполняются единицами. Аналогично заполняются столбцы для остальных вершин (табл. 2.14).

Таблица 2.14. Матрица достижимости задачи поиска максимального числа

	1	2	3	4	5	6	7	8	9	10
1										
2	1	1	1	1	1	1	1	1	1	
3	1	1	1	1	1	1	1	1	1	
4	1	1	1	1	1	1	1	1	1	
5	1	1	1	1	1	1	1	1	1	
6	1	1	1	1	1	1	1	1	1	
7	1	1	1	1	1	1	1	1	1	
8	1	1	1	1	1	1	1	1	1	
9	1	1	1	1	1	1	1	1	1	
10	1	1	1	1	1	1	1	1	1	

Выделенные диагональные элементы матрицы определяют номера столбцов-вершин, которые входят в состав циклических маршру-

тов. Идентичные строки матрицы определяют номера столбцов-вершин, которые входят в состав ациклических маршрутов.

Элементы матрицы достижимости, выделенные черным цветом, соответствуют маршруту m_3 . Элементы, помеченные единицами полужирным курсивом белого и черного цвета, соответствуют маршруту m_4 . Все строки матрицы получились идентичными, так как все вершины графа входят в состав ациклических маршрутов m_1 и m_2 .

Проведем оценку алгоритмической сложности по третьему критерию. В соответствии с третьим критерием необходимо выделить все реально возможные маршруты управления:

$$m_1: 1-2-\underline{3}-4-\underline{5}-6-8-\underline{9}-10; p_1 = 3;$$

$$m_2: 1-2-\underline{3}-7-8-\underline{9}-10; p_2 = 2;$$

$$m_3: 1-2-\underline{3}-4-\underline{5}-2-\underline{3}-4-\underline{5}-6-8-\underline{9}-10; p_3 = 5;$$

$$m_4: 1-2-\underline{3}-7-8-\underline{9}-2-\underline{3}-4-\underline{5}-2-\underline{3}-4-\underline{5}-6-8-\underline{9}-10; p_4 = 7;$$

$$m_5: 1-2-\underline{3}-4-\underline{5}-2-\underline{3}-4-\underline{5}-6-8-\underline{9}-2-\underline{3}-7-8-\underline{9}-10; p_5 = 7$$

$$m_6: 1-2-\underline{3}-7-8-\underline{9}-2-\underline{3}-7-8-\underline{9}-10; p_6 = 4.$$

Оценку структурной сложности программы рассчитываем по следующему соотношению:

$$S_3 = p_1 + p_2 + p_3 + p_4 + p_5 + p_6 = 3 + 2 + 5 + 7 + 7 + 4 = 28.$$

Исходя из полученных результатов расчета метрик структурной сложности по первому ($S_1 = 6$), второму ($S_2 = 9$) и третьему ($S_3 = 28$) критериям выделения маршрутов можно сделать вывод, что программа, характеризуемая заданным графом управления, имеет низкую алгоритмическую сложность, так как количество используемых в тексте операторов условий 3, для проверки которых необходимо проверить от 6 до 28 тестовых вариантов исходных данных.

2.8. Задачи для самостоятельного решения

2.8.1. Задачи с разработкой программы

В задачах 1 – 10, предлагаемых для самостоятельного решения, необходимо выполнить следующее с целью оценки алгоритмической сложности:

- разработать алгоритм решения задачи;
- построить граф потока управления;
- сформировать маршруты тестирования в соответствии с критериями 1, 2 и 3;
- определить значение цикломатического числа, характеризующего структурную сложность программ;
- сформировать матрицы смежности и достижимости;
- провести анализ полученных результатов, сформировав содержательные выводы.

Задача 1. Вывести на экран все натуральные числа из диапазона от A до B , сумма цифр которых равна S . При отсутствии чисел с указанными свойствами сформировать сообщение «Требуемых чисел нет». Границы диапазона A и B и заданная сумма цифр S вводятся с клавиатуры.

Задача 2. Вывести на экран все натуральные трехзначные и пятизначные числа из диапазона от A до B , значение которых кратно 13. При отсутствии чисел с указанными свойствами сформировать сообщение «Требуемых чисел нет». Границы диапазона A и B вводятся с клавиатуры.

Задача 3. Вывести на экран все натуральные числа из диапазона от A до B , у которых совпадают старшая и младшая цифры. При отсутствии чисел с указанными свойствами сформировать сообщение «Требуемых чисел нет». Границы диапазона A и B вводятся с клавиатуры.

Задача 4. Вывести на экран для всех натуральных чисел из диапазона от A до B сами числа и суммы цифр, находящихся на нечетных позициях. Номера позиций отсчитываются с единицы, начиная с младшей цифры. Границы диапазона A и B вводятся с клавиатуры.

Задача 5. Вывести на экран для всех натуральных чисел из диапазона от A до B сами числа и суммы трех последних (младших) цифр. Границы диапазона A и B вводятся с клавиатуры. Если диапазон не содержит трехзначных или чисел с большей разрядностью, сформировать сообщение «Диапазон указан ошибочно».

Задача 6. Вычислить и вывести на экран суммы K старших (находящихся слева) цифр натурального числа A . Число A и значение K

вводятся с клавиатуры. Если количество цифр в числе меньше K , сформировать сообщение «Значение числа K слишком велико».

Задача 7. Вывести на экран все натуральные шестизначные числа из диапазона от A до B , у которых совпадают сумма трех младших и трех старших цифр. При отсутствии чисел с указанными свойствами сформировать сообщение «Требуемых чисел нет». Границы диапазона A и B вводятся с клавиатуры.

Задача 8. Вывести на экран все натуральные числа из диапазона от A до B , в записи которых цифра 7 встречается ровно N раз. При отсутствии чисел с указанными свойствами выдать на экран сообщение «Требуемых чисел нет». Границы диапазона A и B и значение N ввести с клавиатуры.

Задача 9. Вывести на экран сумму S ряда чисел:

$$S = \frac{2}{3} + \frac{3}{7} + \frac{4}{11} + \frac{5}{15} + \dots$$

Сумма вычисляется до получения слагаемого, меньшего заданного значения A . Значение A вводится с клавиатуры.

Задача 10. Вычислить суммы S первых N слагаемых последовательности чисел, образуемых по правилу:

$$S = \frac{1}{1} + \frac{2}{3} + \frac{3}{5} + \frac{4}{7} + \dots$$

Сумму вычислить двумя способами: S_1 – суммирование от первого слагаемого до N -го слагаемого, S_2 – суммирование от N -го слагаемого до первого слагаемого. Значение N ввести с клавиатуры. Вывести на экран вычисленные суммы S_1 и S_2 , а также значение модуля разности между ними.